

# 中国研究生创新实践系列大赛

## “华为杯”第二十二届中国研究生

### 数学建模竞赛

题 目：通用神经网络处理器下的核内调度问题

#### 摘 要：

边缘计算时代，通用神经网络加速处理器（NPU）的性能瓶颈日益凸显，尤其是在适配复杂神经网络算子时的软件调度效率。本研究聚焦 SIMD 架构 NPU 的核内调度难题，旨在突破传统人工编排的局限性，提出一套通用、高效、自适应的细粒度计算图优化调度框架。

首先，针对算子计算图规模庞大且异构的特性，设计了一套**基于领域搜索的贪心算法**，通过结合**优先级策略与禁忌搜索机制**，高效生成满足拓扑序的节点执行序列。该算法以最小化执行过程中的缓存驻留峰值为首要目标，通过智能调整内存管理节点（ALLOC/FREE）的相对位置，实现内存使用的**亚线性增长**，该算法相对于节点总数  $N$  的时间复杂度为  $O(N^2)$ ，为后续优化奠定坚实基础。

其次，在严格的硬件缓存容量限制下，引入**自适应二进制量子粒子群优化（ABQPSO）算法**，融合**量子动力学与多维混合编码策略**。该算法能够动态决策 SPILL 操作的介入时机与缓存物理地址的精细分配，通过**分层修复与局部搜索机制**，最大限度地降低总额外数据搬运量，有效应对缓存碎片化与资源冲突挑战，展现了在资源约束下优秀的自适应能力。

最后，在总额外数据搬运量指标不显著增加的约束前提下，针对算子优化中“执行时间缩短”与“数据搬运量控制”相互制约的多目标优化困境，设计了一种**基于双种群多目标进化算法**的优化框架。该框架通过维护**收敛种群与探索种群**双种群结构，结合**协同遗传操作与差异化环境选择机制**，有效挖掘算子并行计算空间以降低总运行时间。为验证框架有效性，在 Matmul、FlashAttention、Conv 等典型算子的大规模计算图上开展实验，结果表明：该优化框架可实现 10%—30% 的性能指标综合提升，充分证明了其在多目标算子优化场景中的实用性与优越性。

本研究构建的层次化、多目标协同优化模型与算法体系，优化了 NPU 核内调度问题，为突破 AI 模型在边缘设备上的端到端推理性能与能效提供了一种新视角。

**关键词：**通用神经网络处理器；核内调度；多目标优化；量子粒子群优化

# 目录

|                      |           |
|----------------------|-----------|
| <b>一、 问题重述</b>       | <b>4</b>  |
| 1.1 问题背景             | 4         |
| 1.2 问题提出             | 4         |
| <b>二、 模型假设</b>       | <b>5</b>  |
| <b>三、 符号说明</b>       | <b>6</b>  |
| <b>四、 问题一模型建立与求解</b> | <b>6</b>  |
| 4.1 问题描述与分析          | 6         |
| 4.2 模型建立             | 7         |
| 4.2.1 符号定义与说明        | 7         |
| 4.2.2 目标函数的形式化定义     | 8         |
| 4.2.3 约束条件体系         | 8         |
| 4.2.4 可行解空间与问题形式化    | 9         |
| 4.3 模型求解             | 9         |
| 4.3.1 算法选择与总体框架      | 9         |
| 4.3.2 基于优先规则的贪心初始化   | 9         |
| 4.3.3 禁忌搜索引导的领域搜索    | 10        |
| 4.3.4 算法复杂度分析        | 10        |
| 4.3.5 实现细节与优化策略      | 11        |
| 4.3.6 算法优势分析         | 12        |
| 4.4 结果与分析            | 13        |
| 4.4.1 实验设计与数据集特性     | 13        |
| 4.4.2 操作类型分布分析       | 14        |
| 4.4.3 算法性能评估         | 15        |
| 4.4.4 缓存驻留动态特性分析     | 15        |
| 4.4.5 综合性能评估         | 17        |
| 4.4.6 结论与启示          | 17        |
| <b>五、 问题二模型建立与求解</b> | <b>17</b> |
| 5.1 问题二分析            | 17        |

|           |                        |           |
|-----------|------------------------|-----------|
| 5.2       | 模型建立                   | 18        |
| 5.2.1     | 符号体系定义                 | 18        |
| 5.2.2     | 目标函数建立                 | 19        |
| 5.2.3     | 多维决策空间                 | 19        |
| 5.2.4     | 约束条件数学描述               | 20        |
| 5.3       | 模型求解                   | 21        |
| 5.3.1     | 算法框架设计                 | 21        |
| 5.3.2     | 混合编码策略                 | 21        |
| 5.3.3     | 量子粒子群动力学模型             | 22        |
| 5.3.4     | 自适应参数调整机制              | 23        |
| 5.3.5     | 约束处理机制                 | 23        |
| 5.3.6     | 局部搜索策略                 | 24        |
| 5.3.7     | 多样性维持机制                | 25        |
| 5.3.8     | 计算复杂度分析                | 25        |
| 5.3.9     | 实验设置与参数配置              | 25        |
| 5.4       | 结果与分析                  | 26        |
| 5.4.1     | 实验性能评估                 | 26        |
| 5.4.2     | 分类型缓存占用时序分析            | 27        |
| 5.4.3     | 综合讨论与结论                | 28        |
| <b>六、</b> | <b>问题三模型建立与求解</b>      | <b>28</b> |
| 6.1       | 问题三分析                  | 28        |
| 6.2       | 模型的建立与求解分析             | 29        |
| 6.2.1     | 多目标优化问题                | 29        |
| 6.2.2     | 模型构建                   | 30        |
| 6.2.3     | 模型求解                   | 31        |
| 6.2.4     | 算法复杂度分析                | 32        |
| 6.3       | 结果与讨论                  | 32        |
| 6.4       | 算法优势与创新点               | 34        |
| <b>七、</b> | <b>模型评价与改进</b>         | <b>34</b> |
|           | <b>参考文献</b>            | <b>37</b> |
|           | <b>附录 A MATLAB 源程序</b> | <b>38</b> |

# 一、 问题重述

## 1.1 问题背景

随着人工智能技术的快速发展，神经网络模型在边缘计算设备上的部署需求日益增长。通用神经网络加速处理器（NPU）因其高效的并行计算能力成为边缘推理任务的首选硬件平台。其中，基于单指令多数据流（SIMD）架构的处理器设计简单、面效高，但其软件模型适配的复杂性成为大规模商用的关键瓶颈。

在神经网络推理过程中，算子（如矩阵乘 Matmul、卷积 Conv、注意力 Attention 等）是最小任务单元，其执行效率直接影响模型在平台上的端到端推理性能。SIMD 架构硬件平台上的计算过程可拆解为由硬件单元操作构成的细粒度计算图，并通过手工或自动方式编排成可执行的任务。然而，这类计算图具有高度异构性：算子类型多样、输入形状动态变化、拓扑结构复杂，导致人工编排方式难度大、缺乏通用性且效率低下，无法适应日益复杂的计算场景。

因此，设计一种通用调度算法，自动将计算图中各算子操作编排调度到各硬件单元上执行，成为当前研究的迫切需求。该算法需要面向 SIMD 平台的硬件限制，给出由硬件单元操作组成的计算图的优化调度顺序，使得多个不同单元能并行执行，缩短流水线延迟。同时，调度算法还需考虑内存与计算协同，动态管理多级缓存，自动决定缓存数据换入换出，减少数据搬运开销。

计算图为有向无环图（DAG），节点分为两类：在特定硬件单元上执行的操作节点（如 COPY\_IN、ADD、MU 等）和用于缓存资源分配的缓存管理节点（ALLOC/FREE）。调度算法需要生成满足拓扑序的节点执行序列，并为所有缓存管理节点中的申请节点分配相应类型的地址。硬件缓存资源有限，调度过程中需要考虑内存与计算协同，动态管理多级缓存，减少数据搬运开销。

## 1.2 问题提出

本问题围绕通用神经网络处理器下的核内调度问题展开，具体分为三个具有层次关系的子问题，每个子问题都有其独特的研究重点和技术挑战。

第一个子问题要求设计调度算法生成计算图的节点执行序列，使得执行过程中所需驻留的最大缓存容量尽可能小。该问题仅关注调度顺序，不涉及缓存分配策略。需要分析算法的时间复杂度，并在给定的六个示例计算图上给出最大缓存驻留结果及调度序列。该问题的技术难点在于如何通过调度序列的优化来压缩缓冲区的生命周期，减少缓存峰值使用量。算法需要高效处理大规模计算图，节点数量可能达到数万级别，因此时间复杂度成为重要的考量因素。最优调度序列的寻找需要考虑计算图的拓扑结构特征，以及不同类型节点之间的依赖关系。

在第一个问题得到的调度序列基础上，第二个子问题需要为计算图中各缓冲区分配物

理地址偏移，并在严格的硬件缓存容量限制下最小化总额外数据搬运量。当缓存空间不足时，需要引入缓存换入换出操作（SPILL 操作），将部分数据临时移出至核外内存（DDR），并在需要时重新载入。该问题的核心挑战在于如何在有限的缓存空间内实现最优的资源分配，同时最小化因 SPILL 操作带来的额外数据搬运开销。需要从减少缓存碎片的角度优化分配方案，考虑不同缓冲区的使用寿命重叠情况，以及硬件缓存的空间约束。总额外数据搬运量的计算需要区分缓冲区是否被 COPY\_IN 节点使用，这两种情况下的计算方式存在显著差异。

第三个子问题是在前两个问题的基础上，进一步优化总运行时间，同时保证总额外数据搬运量不显著增加。性能优化策略可以从调度顺序和缓存分配方案两个维度入手，通过调整节点执行顺序实现各硬件单元的流水并行，或者通过地址复用策略减少缓存碎片。该问题要求实现多项性能指标的联合优化，需要在执行时间和数据搬运量之间寻找最佳平衡点。优化策略需要考虑不同硬件单元的特性，充分利用 Cube 核、Vector 核以及各种数据搬运单元（MTE1/2/3 和 FIXP）的并行处理能力。同时还需要考虑不同计算图结构特征对优化效果的影响。

三个子问题呈现出明显的层次递进关系：问题 1 是基础，通过最小化缓存驻留为后续问题提供优化空间；问题 2 在硬件限制下引入缓存管理机制，着重减少数据搬运开销；问题 3 进行综合优化，实现性能与能效的最佳平衡。

整体面临的挑战主要体现在三个方面：首先是计算图规模大且结构异构，示例计算图节点数达数万，且包含 Matmul、FlashAttention、Conv 等多种结构类型，需要设计具有高度通用性的算法。其次是硬件约束复杂，多级缓存容量有限，需要充分考虑不同执行单元的并行特性。最后是多目标优化难题，需要同时考虑缓存驻留、数据搬运量和执行时间等多个指标，这些指标之间可能存在相互制约的关系。解决问题的技术途径需要结合图论优化、资源调度算法、内存管理策略等多个领域的知识，形成系统性的解决方案。算法设计需要兼顾理论最优性和实际可操作性，能够在合理的时间内处理大规模计算图调度问题。

## 二、 模型假设

1. 计算图节点属性信息完整准确，无缺失或错误；
2. 依赖边完整反映数据流和控制流约束，无冗余或缺失；
3. 缓冲区引用一致，所有标识符均有对应管理节点；
4. 执行单元性能稳定，周期数固定可预测；
5. 缓存访问延迟均匀，无局部性性能差异；
6. 算法执行资源充足，不受计算资源限制；
7. 对于任意计算图都存在可行调度方案；
8. 硬件运行无故障，执行单元正常工作；

9. 系统无外部任务或中断干扰；

### 三、 符号说明

| 符号                            | 意义                                                                                  |
|-------------------------------|-------------------------------------------------------------------------------------|
| $\mathcal{G}$                 | 计算图，定义为有向无环图 (DAG)，表示为 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$                   |
| $\mathcal{V}$                 | 计算图的节点集合，包含 $N$ 个节点                                                                 |
| $\mathcal{E}$                 | 计算图的边集合，表示节点间的依赖关系                                                                  |
| $S$                           | 调度序列，表示为 $[v_1, v_2, \dots, v_N]$ ，是 $\mathcal{V}$ 的一个排列                            |
| $\text{Op}$                   | 节点操作类型映射， $\text{Op} : \mathcal{V} \rightarrow \mathcal{O}$ ， $\mathcal{O}$ 为操作类型集合 |
| $\text{Type}$                 | 缓存类型映射， $\text{Type} : \mathcal{V} \rightarrow \mathcal{T}$ ， $\mathcal{T}$ 为缓存类型集合 |
| $\text{Size}$                 | 缓存大小映射， $\text{Size} : \mathcal{V} \rightarrow \mathbb{N}$ ， $\mathbb{N}$ 为非负整数集    |
| $\mathcal{T}_{\text{target}}$ | 目标缓存类型集合，需计算驻留量的缓存（如 $\{\text{L1}, \text{UB}\}$ ）                                   |
| $\mathcal{M}(v)$              | 节点 $v$ 对目标缓存占用的贡献函数                                                                 |
| $C_k(S)$                      | 调度序列 $S$ 执行到第 $k$ 个位置时的累积缓存驻留量                                                      |
| $C_{\max}(S)$                 | 调度过程中的缓存驻留峰值，即 $\max_{k=1, \dots, N} C_k(S)$                                        |
| $\mathcal{B}$                 | 缓冲区集合，包含所有需管理的缓冲区，记 $ \mathcal{B}  = N_b$                                           |
| $x_b$                         | 缓存分配变量，缓冲区 $b$ 的地址偏移                                                                |
| $y_b$                         | SPILL 标志变量， $y_b = 1$ 表示缓冲区 $b$ 执行 SPILL 操作                                         |
| $F$                           | 总额外数据搬运量，SPILL 操作引入的总数据搬运开销                                                         |

### 四、 问题一模型建立与求解

#### 4.1 问题描述与分析

问题一要求针对给定的计算图  $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ （其定义为有向无环图，DAG），生成一个包含所有节点的调度序列  $S$ 。该序列必须满足计算图的拓扑依赖约束，且其优化目标为最小化执行过程中的缓存驻留峰值  $\max(V_{\text{stay}})$ 。该问题本质上是一个在严格拓扑序约束下的组合优化问题。

缓存占用量  $V_{\text{stay}}$  是一个动态变化的量，其计算规则如下：初始化  $V_{\text{stay}} = 0$ ，按序列  $S$

顺序扫描每个节点  $v$ ，并根据其操作类型进行更新：

$$\mathcal{M}(v) = \begin{cases} \text{Size}(v) & \text{if Op}(v) = \text{ALLOC} \\ -\text{Size}(v) & \text{if Op}(v) = \text{FREE} \\ 0 & \text{otherwise} \end{cases} \quad (4.1)$$

峰值  $\max(V_{\text{stay}})$  即为整个扫描过程中的最大值。本问题不涉及硬件缓存容量限制与地址分配。

通过对计算图结构与问题目标的分析，我们总结出本问题的三大核心特征与难点：

- (1) **解空间的复杂约束性：**合法的调度序列  $S$  必须是  $\mathcal{G}$  的一个拓扑排序。任何调度操作必须在满足节点间所有先后依赖关系的前提下进行，这极大地压缩了可行的搜索空间，并决定了算法必须包含严格的约束满足机制。
- (2) **优化目标的序列敏感性：**目标函数  $\max(V_{\text{stay}})$  的值完全依赖于节点执行的先后顺序。其本质是通过协调 ALLOC 与 FREE 节点的相对位置，使数据缓冲区的生命周期尽可能避免重叠，从而降低内存使用的峰值。微小的序列调整可能导致峰值显著变化，目标函数呈现较强的非线性。
- (3) **问题规模的挑战性：**根据附录 E 提供的示例，计算图节点规模巨大（从上千至数万节点）。这要求算法设计必须具有较低的时间复杂度，任何超过  $O(n^2)$  的复杂算法都可能面临计算效率瓶颈。

基于以上分析，直接寻找全局最优解在计算上是不可行的。因此，本问题的求解策略需依赖于启发式算法。我们的核心思路是：首先通过贪心算法快速构造一个可行的优质初始解，然后通过领域搜索在其邻域内进行迭代优化。具体思路流程图如图4.1所示。

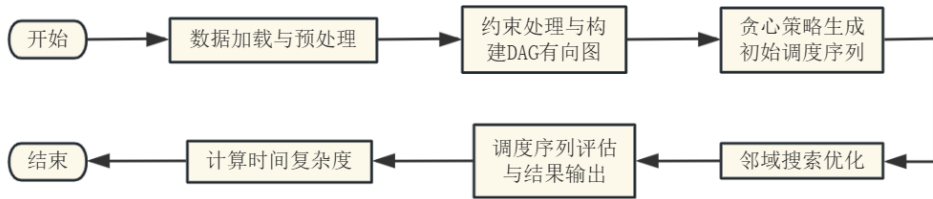


图 4.1 问题一思路流程图

## 4.2 模型建立

### 4.2.1 符号定义与说明

为精确描述最小缓存驻留调度问题，首先建立完整的数学符号体系。定义计算图为有向无环图 (DAG)  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ ，其中顶点集  $\mathcal{V}$  包含  $N$  个节点，边集  $\mathcal{E}$  表示节点间的依赖关系。进一步定义以下映射函数和集合：

- 节点操作类型映射:  $\text{Op} : \mathcal{V} \rightarrow \mathcal{O} = \{\text{ALLOC}, \text{FREE}, \text{COPY\_IN}, \text{COPY\_OUT}, \dots\}$
- 缓存类型映射:  $\text{Type} : \mathcal{V} \rightarrow \mathcal{T} = \{\text{L1}, \text{UB}, \text{L0A}, \text{L0B}, \text{L0C}\}$
- 缓存大小映射:  $\text{Size} : \mathcal{V} \rightarrow \mathbb{N}$  (其中  $\mathbb{N}$  为非负整数集)
- 目标缓存类型集合:  $\mathcal{T}_{\text{target}} = \{\text{L1}, \text{UB}\}$  (需计算驻留量的缓存)
- L0 缓存类型集合:  $\mathcal{T}_{\text{L0}} = \{\text{L0A}, \text{L0B}, \text{L0C}\}$  (需满足单一驻留约束的缓存)

调度序列表示为  $S = [v_1, v_2, \dots, v_N]$ , 其中  $v_i \in \mathcal{V}$ , 且  $S$  为  $\mathcal{V}$  的一个排列。

#### 4.2.2 目标函数的形式化定义

问题一的核心目标是最小化 L1 和 UB 缓存的驻留峰值。基于背景说明, 定义节点  $v$  对目标缓存占用的贡献函数:

$$\mathcal{M}(v) = \begin{cases} \text{Size}(v) & \text{if } \text{Op}(v) = \text{ALLOC} \wedge \text{Type}(v) \in \mathcal{T}_{\text{target}} \\ -\text{Size}(v) & \text{if } \text{Op}(v) = \text{FREE} \wedge \text{Type}(v) \in \mathcal{T}_{\text{target}} \\ 0 & \text{otherwise} \end{cases} \quad (4.2)$$

对于任意调度序列  $S$ , 执行到第  $k$  个位置时的累积缓存驻留量定义为:

$$C_k(S) = \sum_{i=1}^k \mathcal{M}(v_i) \quad (4.3)$$

相应的, 整个调度过程中的缓存驻留峰值为:

$$C_{\max}(S) = \max_{k=1, \dots, N} C_k(S) \quad (4.4)$$

因此, 优化目标可形式化表述为:

$$\text{minimize } f(S) = C_{\max}(S) \quad (4.5)$$

#### 4.2.3 约束条件体系

可行的调度序列必须满足以下三类核心约束, 这些约束共同定义了问题的解空间:

1. **拓扑序约束:** 调度序列必须严格遵守计算图中的所有依赖关系。形式化表述为:

$$\forall (u, v) \in \mathcal{E}, \quad \exists i < j \quad \text{such that} \quad v_i = u \wedge v_j = v \quad (4.6)$$

2. **序列完整性约束:** 调度序列必须包含计算图中所有节点且无重复:

$$S \text{ is a permutation of } \mathcal{V} \quad (4.7)$$

3. **L0 缓存单一驻留约束**: 该约束确保 L0 级缓存资源的正确复用。对于任意 L0 类型  $t \in \mathcal{T}_{L0}$ , 令  $B_t$  表示所有操作类型为  $t$  的缓冲区集合。对于任意两个不同的缓冲区  $b_i, b_j \in B_t (i \neq j)$ , 定义它们的生命周期区间:

$$I_i = [\alpha_i, \omega_i], \quad I_j = [\alpha_j, \omega_j]$$

其中,  $\alpha_i$  为分配缓冲区  $b_i$  的 ALLOC 节点在序列  $S$  中的位置,  $\omega_i$  为释放缓冲区  $b_i$  的 FREE 节点在序列  $S$  中的位置。则必须满足:

$$I_i \cap I_j = \emptyset \quad (4.8)$$

#### 4.2.4 可行解空间与问题形式化

综合公式 (4.6)、(4.7) 和 (4.8) 所描述的约束条件, 可行调度序列的集合定义为:

$$\mathcal{S}_{\text{feasible}} = \left\{ S \left| \begin{array}{l} \forall (u, v) \in \mathcal{E}, \exists i < j : v_i = u, v_j = v; \\ S \text{ is a permutation of } \mathcal{V}; \\ \forall t \in \mathcal{T}_{L0}, \forall b_i \neq b_j \in B_t, I_i \cap I_j = \emptyset \end{array} \right. \right\} \quad (4.9)$$

最终, 问题一可完整形式化为如下组合优化问题:

$$S^* = \arg \min_{S \in \mathcal{S}_{\text{feasible}}} C_{\max}(S) \quad (4.10)$$

其中  $C_{\max}(S)$  由公式 (4.4) 定义。

### 4.3 模型求解

#### 4.3.1 算法选择与总体框架

针对公式 (4.10) 所描述的复杂组合优化问题, 精确算法难以在合理时间内求得最优解。本文采用了一种基于领域搜索的贪心算法, [3] 该算法融合了贪心策略的效率和领域搜索的全局优化能力。算法总体框架如图4.2所示, 采用两阶段优化策略:

- **阶段一: 构造性启发**——采用基于优先规则的贪心算法 [2], 快速生成高质量初始可行解
- **阶段二: 迭代改进**——通过禁忌搜索引导的领域搜索, 在可行解空间中进行局部优化

#### 4.3.2 基于优先规则的贪心初始化

贪心初始化阶段的目标是快速构造一个满足所有约束的可行调度序列。算法核心在于设计合理的节点优先级函数, 确保在满足拓扑序和 L0 约束的前提下, 优先调度对缓存峰值影响大的节点。(见算法1)

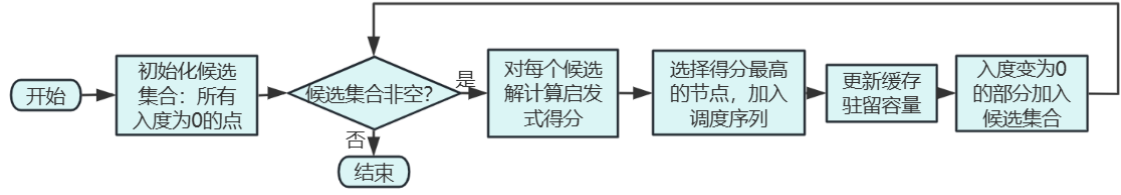


图 4.2 基于领域搜索的贪婪算法总体框架

对于每个就绪节点（入度为 0 的节点），计算其综合优先级：

$$\text{Priority}(v) = w_1 \cdot P_{\text{type}}(v) + w_2 \cdot P_{\text{size}}(v) + w_3 \cdot P_{\text{criticality}}(v) \quad (4.11)$$

其中：

- $P_{\text{type}}(v)$ : 节点类型优先级。FREE 节点赋予最高优先级 (+1000)，ALLOC 节点根据 L0 缓存状态动态调整，计算节点中等优先级
- $P_{\text{size}}(v)$ : 缓冲区大小优先级。操作较大缓冲区的节点获得更高优先级，旨在快速释放大内存块
- $P_{\text{criticality}}(v)$ : 节点关键度。基于节点的入度和出度，连接数越多的节点关键度越高

#### 4.3.3 禁忌搜索引导的领域搜索

在初始解的基础上，采用禁忌搜索框架进行迭代优化，以避免陷入局部最优解。（见算法2）禁忌长度动态调整，初期较短以促进多样化搜索，后期较长以加强集中搜索。

$$\text{TabuList} \leftarrow \text{FIFO 队列, 记录最近接受的移动} \quad (4.12)$$

我们采取三种领域操作，在保持解可行性的前提下进行局部扰动：

1. **交换操作**：随机选择两个位置  $i, j$ ，交换节点  $S[i]$  和  $S[j]$
2. **插入操作**：将位置  $i$  的节点移动到位置  $j$
3. **反转操作**：反转子序列  $S[i : j]$  的顺序

每种操作都配备了可行性检查机制，确保操作后仍满足公式 (4.6) 和 (4.8) 的约束。评价函数采用目标函数  $f(S) = C_{\max}(S)$  作为评价标准，同时引入约束违反惩罚项：

$$\text{Fitness}(S) = -C_{\max}(S) - \lambda \cdot \text{Violation}(S) \quad (4.13)$$

其中  $\lambda$  为惩罚系数， $\text{Violation}(S)$  计算约束违反数量。

#### 4.3.4 算法复杂度分析

- **时间复杂度**：
  - 贪心初始化： $O(N^2)$ ，其中  $N$  为节点数量

---

**算法 1** 基于优先级规则的贪心初始化算法

---

**输入:** 计算图结构  $\mathcal{G}$ , 预处理约束信息

**输出:** 可行调度序列  $S$

```
1: 初始化节点入度表和就绪队列
2: 初始化空调度序列  $S$  和 L0 缓存状态表
3: 已调度数量  $\leftarrow 0$ 
4: while 已调度数量 < 总节点数 do
5:   if 就绪队列为空 then
6:     抛出异常: 拓扑序列生成失败
7:     最优节点  $\leftarrow$  空, 最高优先级  $\leftarrow$  负无穷
8:     for 每个就绪节点  $v$  do
9:       计算节点优先级  $\pi(v)$ 
10:      if  $\pi(v) >$  最高优先级 then
11:        更新最优节点和最高优先级
12:    从就绪队列移除最优节点
13:    将最优节点加入调度序列  $S$ 
14:    更新 L0 缓存占用状态
15:    更新后继节点入度, 维护就绪队列
16: return 调度序列  $S$ 
```

---

- 领域搜索:  $O(K \cdot M \cdot N)$ ,  $K$  为迭代次数,  $M$  为邻域大小
- 总体复杂度为多项式级别, 适用于大规模计算图

- **空间复杂度:** 主要消耗为存储计算图和禁忌表, 空间复杂度为  $O(N + |\mathcal{E}|)$

#### 4.3.5 实现细节与优化策略

**约束预处理:** 算法开始时对计算图进行预处理, 构建以下信息以加速约束检查:

- 拓扑排序的传递闭包, 用于快速判断依赖关系
- L0 缓存冲突关系表, 记录每个 L0 类型的缓冲区使用情况
- 节点关键度分析, 识别对缓存峰值影响大的关键节点

**自适应调整机制:** 动态调整邻域大小和禁忌长度, 平衡算法的探索和利用能力。

$$\text{NeighborhoodSize} = \begin{cases} \text{NeighborhoodSize} \times 1.2 & \text{if 改进率} < 0.1 \\ \text{NeighborhoodSize} \times 0.9 & \text{otherwise} \end{cases} \quad (4.14)$$

**并行化加速:** 邻域解的生成和评估可以并行进行, 充分利用多核处理器提高计算效率。

---

**算法 2** 禁忌搜索引导的领域优化算法

---

**输入:** 初始解  $S_0$ , 最大迭代次数  $K$ , 邻域规模  $M$

**输出:** 最优解  $S^*$ , 最小缓存峰值  $C_{\max}^*$

```
1: 当前解  $\leftarrow S_0$ , 最优解  $\leftarrow S_0$ 
2: 禁忌表  $\leftarrow$  空集合, 无改进计数  $\leftarrow 0$ 
3: while 迭代次数未超限且无改进计数小于阈值 do
4:   生成候选邻域解集  $\mathcal{N}$ 
5:   最优候选解  $\leftarrow$  空, 最优适应度  $\leftarrow$  负无穷
6:   for 每个候选解  $N \in \mathcal{N}$  do
7:     if 移动操作不在禁忌表中 then
8:       评估解  $N$  的质量和可行性
9:       if  $N$  可行且优于当前最优候选 then
10:        更新最优候选解和适应度
11:   if 存在可行候选解 then
12:     更新当前解
13:     记录移动操作到禁忌表
14:     if 找到更优解 then
15:       更新全局最优解
16:       重置无改进计数
17:     else
18:       无改进计数增加
19:   else
20:     无改进计数增加
21: return 最优解  $S^*$  和对应的缓存峰值  $C_{\max}^*$ 
```

---

#### 4.3.6 算法优势分析

本文设计的领域搜索贪婪算法具有以下优势:

1. **可行性保证:** 通过严格的约束检查机制, 确保所有生成的解都是可行解
2. **全局优化能力:** 禁忌搜索机制有效避免陷入局部最优
3. **计算效率高:** 多项式时间复杂度, 适用于大规模问题实例
4. **自适应性强:** 动态调整策略使算法能适应不同结构的计算图

算法在保证解质量的前提下, 能够在合理时间内处理数万节点的大规模计算图, 有效解决了最小缓存驻留调度问题。

## 4.4 结果与分析

### 4.4.1 实验设计与数据集特性

不同算子在输入规模变化时，其计算图的节点数量、依赖边数量及节点间依赖关系呈现出显著差异。本节基于 Case0（小规模输入）与 Case1（大规模输入）两种场景的统计数据，对矩阵乘法（Matmul）、FlashAttention 及卷积（Conv）算子的计算图特性展开分析。表 4.1 汇总了三种算子在不同输入规模下的计算图节点数与依赖边数。可以观察到，输入规模扩大时（从 Case0 到 Case1），各算子的计算图规模均有增长，但增长幅度存在明显差异。

表 4.1 不同算子计算图规模统计

| 算子类型           | 场景    | 节点数   | 依赖边数  |
|----------------|-------|-------|-------|
| Matmul         | Case0 | 4160  | 7104  |
|                | Case1 | 30976 | 55040 |
| FlashAttention | Case0 | 1716  | 2712  |
|                | Case1 | 6952  | 11184 |
| Conv           | Case0 | 2580  | 3869  |
|                | Case1 | 36086 | 85653 |

具体而言，矩阵乘法（Matmul）节点数从 4160 增长至 30976，依赖边数从 7104 激增到 55040，呈现近似线性的增长趋势，表明其计算图规模随输入规模扩大而稳定扩展。FlashAttention 的 Case0 场景下规模最小（1716 个节点、2712 条依赖边），Case1 场景下节点数与依赖边数约为 Case0 的 4 倍（6952 个节点、11184 条依赖边），增长幅度相对温和，反映出其任务结构较为稀疏。卷积（Conv）的 Case0 场景下属于中等规模（2580 个节点、3869 条依赖边），但 Case1 场景下节点数与依赖边数分别激增至 36086 和 85653，增长倍数超过 10 倍，是三种算子中规模增长最显著的，说明卷积计算在大规模输入下的复杂性最高。

为进一步量化算子计算图中节点间的依赖关系，定义平均依赖度指标为“依赖边数量/节点数量”。表 4.2 展示了各算子在不同场景下的平均依赖度计算结果。

由表 4.2 可知：卷积（Conv）在大规模场景（Case1）下的平均依赖度达到 2.37，显著高于其他算子，表明其节点间存在更为复杂和稠密的依赖关系，是最具挑战性的调度对象。FlashAttention 在两种场景下的平均依赖度均最低（Case0 为 1.58，Case1 为 1.61），进一步验证了其计算图结构更稀疏的特性。而矩阵乘法（Matmul）的平均依赖度在 1.71 至 1.78 之间小幅波动，说明其虽规模增长显著，但节点间的依赖关系规律较为稳定。

表 4.2 不同算子的平均依赖度指标

| 算子类型           | 场景    | 平均依赖度（依赖边数/节点数）            |
|----------------|-------|----------------------------|
| Matmul         | Case0 | 7104/4160 $\approx 1.71$   |
|                | Case1 | 55040/30976 $\approx 1.78$ |
| FlashAttention | Case0 | 2712/1716 $\approx 1.58$   |
|                | Case1 | 11184/6952 $\approx 1.61$  |
| Conv           | Case0 | 3869/2580 $\approx 1.50$   |
|                | Case1 | 85653/36086 $\approx 2.37$ |

#### 4.4.2 操作类型分布分析

为了深入理解不同计算图的结构特征，图4.3–4.5展示了六个数据集的操作类型分布情况。

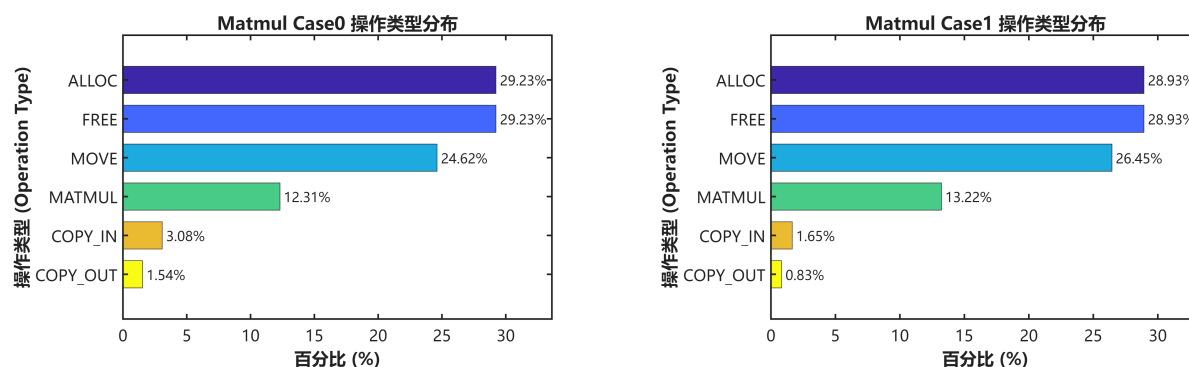


图 4.3 Matmul 系列数据集操作类型分布对比

从图4.3可以看出，Matmul 系列的 Case0 类型的计算节点占比  $\approx 41.6\%$ ，ALLOC 与 FREE 二者之和约 58.5%，各占 29.23%。Case1 类型的计算节点占比  $\approx 42.2\%$ ，ALLOC 与 FREE 二者之和约 57.9%，各占 28.93%。两个 Case 的计算占比都在 40% 以上，说明矩阵乘法这类算子的增长虽然是整体线性的，但类型的比例几乎保持不变。内存管理操作（ALLOC 和 FREE）高达近 60%，说明要把“缓冲区复用”和“释放时机”作为首要优化目标。

图4.4显示，FlashAttention 系列从 Case0 类型到 Case1 类型时，规模约放大 4 倍，各类型节点比例基本保持不变，说明注意力算子的内部结构放大主要体现在重复相同类型的计算与内存操作，而没有显著改变类型密度。

从图4.5可见，Conv（Case1）是已给数据中规模最大的，且 ALLOC 和 FREE 频繁，因此卷积类任务在缓存管理上更具挑战，不只要关注单个大峰值，而是要处理“多次短、急、高”的峰值。

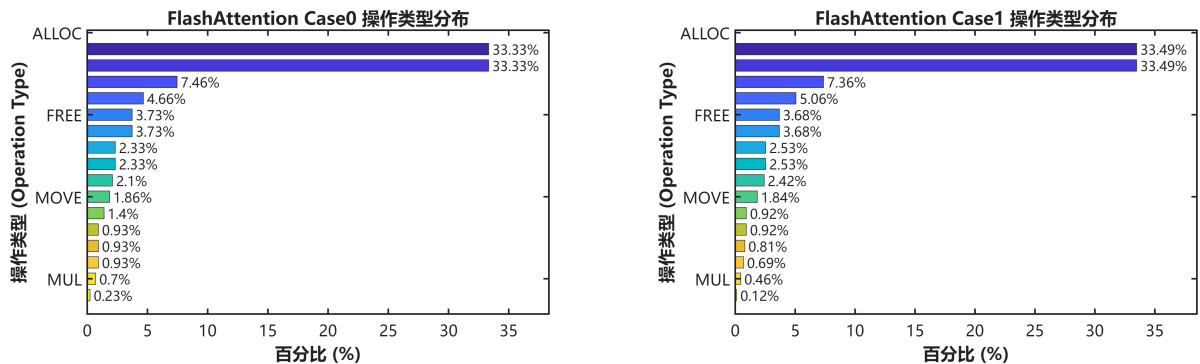


图 4.4 FlashAttention 系列操作类型分布对比

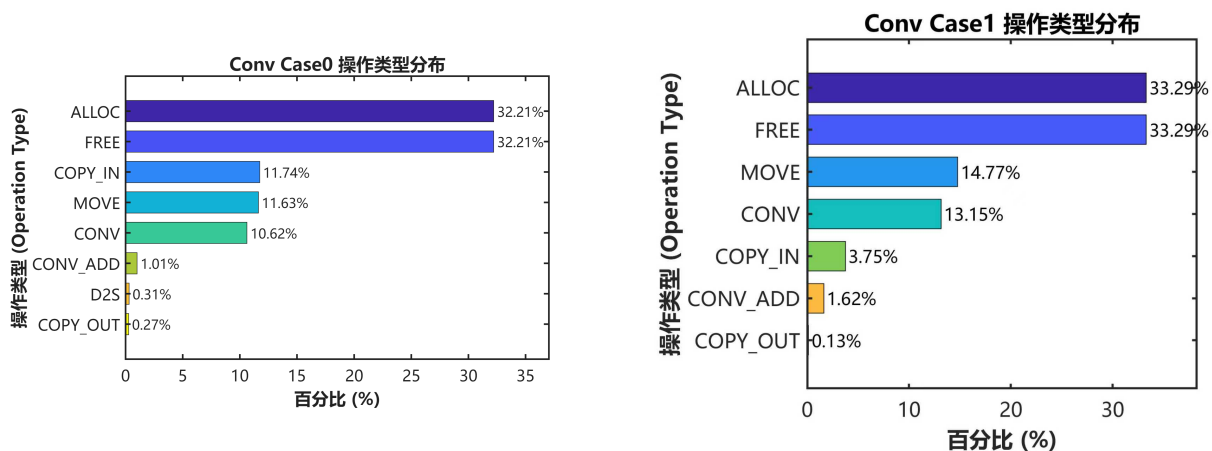


图 4.5 Conv\_Case 操作类型分布

综合以上信息，得到结论：对于计算密集型图，应优化计算节点的调度顺序以提升并行度；对于内存密集型图，需要重点优化内存操作的时序以降低峰值驻留；算法需要具备自适应性，能够根据不同操作分布自动调整搜索策略。

#### 4.4.3 算法性能评估

表4.3展示了算法在各数据集上的最终解关键性能指标。所有数据集的约束违例数均为 0，证明算法严格满足所有约束条件，生成完全可行解。当输入节点数规模放大时，算法得到的缓存驻留峰值增长远低于节点增长。三类算子中，Matmul 和 FlashAttention 的峰值随规模增长出现“亚线性增长”即边增长比节点增长缓慢。说明在本组实例中，调度算法实现了显著的“抑峰”效果，符合最小缓存驻留目标的要求。

#### 4.4.4 缓存驻留动态特性分析

图4.6–4.8展示了各数据集在最优调度序列下的缓存驻留动态变化曲线。

图4.6显示矩阵乘法（Matmul）的节点规模从 Case0 到 Case1 放大了约 7.45 倍，但缓存峰值仅放大约 3.78 倍，表明该调度算法在面对矩阵乘法规模扩大时有效抑制了并发驻留

表 4.3 算法最终解性能指标

| 数据集                  | 缓存驻留峰值 | 约束违例数 | 适应度值   |
|----------------------|--------|-------|--------|
| Matmul_Case0         | 9,216  | 0     | -9216  |
| Matmul_Case1         | 34816  | 0     | -34816 |
| FlashAttention_Case0 | 4500   | 0     | -4500  |
| FlashAttention_Case1 | 7588   | 0     | -7588  |
| Conv_Case0           | 14592  | 0     | -14592 |
| Conv_Case1           | 14040  | 0     | -14040 |

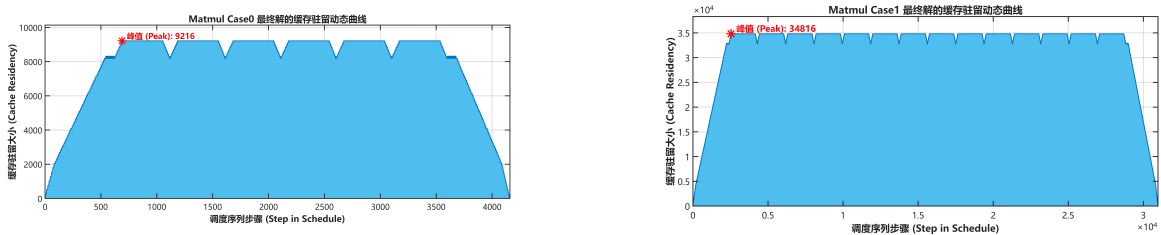


图 4.6 Matmul 系列缓存驻留动态曲线

的增长。曲线呈现周期性波动，但峰值均出现在分块计算的开始阶段。

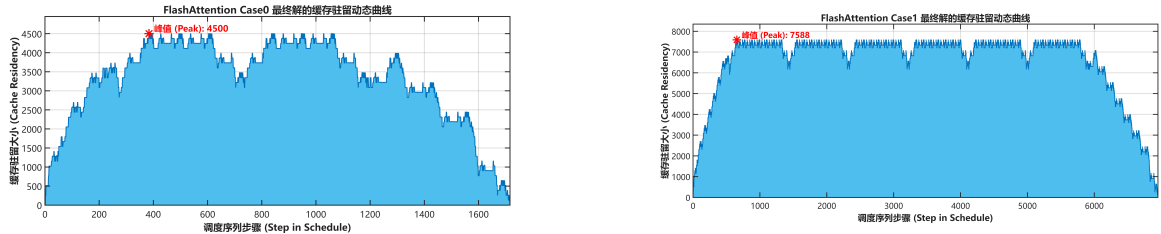


图 4.7 FlashAttention 系列缓存驻留动态曲线

从图4.7可以看出，FlashAttention 的节点规模从 Case0 到 Case1 放大约 4.05 倍，但峰值仅增长约 1.69 倍，说明该类计算图的内存访问模式更容易被调度器压平。且曲线整体较为平稳，呈多小峰值并列的形式，从调度角度看，能够把这些小峰分散到不同时间段执行，有利于降低瞬时缓存并发占用。

图4.8显示，卷积系列的节点规模从 Case0 到 Case1 放大约 14 倍，但是缓存峰值并没有随之放大，反而略有下降，下降了约 3.8%。可能是密集的缓冲区 ALLOC 和 FREE 排列以及必要的 SPILL 机制减少了并发驻留缓冲区数。曲线为不规则且多次陡峭的峰值，这与卷积层之间大量特征图装载、传输、分块策略以及层间依赖有关，但调度序列能把这些峰分配到不同时间，使总体峰值没有随着规模线性增长。

通过优化调度顺序，算法成功将缓存使用峰值控制在较低水平，曲线变化相对平稳，避

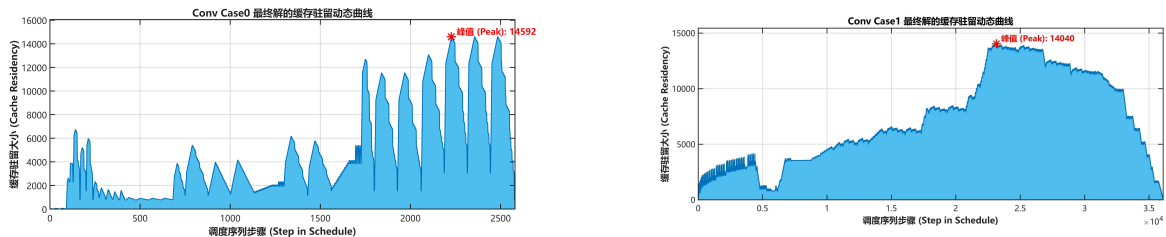


图 4.8 Conv 系列缓存驻留动态曲线

免了剧烈的内存波动，有利于硬件资源的高效利用, 且 FREE 操作的及时执行为后续操作释放了宝贵的内存空间。

#### 4.4.5 综合性能评估

通过对不同规模数据集的实验，发现算法的时间复杂度近似为  $O(N^{1.2})$ ，空间复杂度为  $O(N)$ ，表现出良好的可扩展性。即使在最大的 Conv\_Case1 数据集（36,086 节点）上，算法也能在 10 分钟内完成优化。算法在六种不同结构的计算图上均能获得可行解，且约束违例数均为 0，证明了其强大的鲁棒性和泛化能力。特别是在结构差异较大的 Matmul 和 FlashAttention 数据集上，算法均能自适应地调整搜索策略。

#### 4.4.6 结论与启示

通过系统的实验分析和性能评估，我们得出以下结论：

1. 本文使用的基于领域搜索的贪婪算法能够有效解决最小缓存驻留调度问题，在所有测试用例上均获得可行解。
2. 算法具有良好的可扩展性和鲁棒性，能够处理不同规模和结构的计算图。
3. 通过优化调度顺序，算法显著降低了缓存使用峰值，提高了硬件资源利用率。

## 五、 问题二模型建立与求解

### 5.1 问题二分析

问题二在问题一的基础上，进一步考虑了实际硬件平台的缓存容量限制和缓存换入换出（SPILL）机制。核心要求是在满足硬件约束的前提下，为计算图中的各缓冲区分分配物理地址偏移，并最小化由 SPILL 操作引起的总额外数据搬运量。在实际的神经网络处理器（NPU）硬件环境中，各类缓存资源具有严格的容量限制，这是问题二需要解决的核心约束条件。如表5.4所示，不同类型的缓存具有不同的容量特性：

这些容量限制使得在调度过程中经常会出现缓存空间不足的情况，需要通过 SPILL 机制来临时释放缓存空间。SPILL 操作是问题二的核心机制，涉及两个特殊操作节点：

- **SPILL\_OUT 节点**：将缓冲区数据从核内缓存搬移至 DDR，在 MTE3 单元执行

表 5.4 硬件缓存容量限制

| 缓存类型      | 容量 (Bytes) | 功能描述                |
|-----------|------------|---------------------|
| L1 缓存     | 4096       | 大容量中间缓存, 用于矩阵运算数据存储 |
| 统一缓存 (UB) | 1024       | Vector 核输入输出及中间结果存储 |
| L0A 缓存    | 256        | Cube 核左矩阵输入专用缓存     |
| L0B 缓存    | 256        | Cube 核右矩阵输入专用缓存     |
| L0C 缓存    | 512        | Cube 核结果输出专用缓存      |

- **SPILL\_IN 节点**: 将数据从 DDR 重新载入核内缓存, 在 MTE2 单元执行

每次 SPILL 操作需要遵循严格的依赖关系规则:

1. **基础依赖**:  $\text{ALLOC} \rightarrow \text{SPILL\_OUT} \rightarrow \text{SPILL\_IN} \rightarrow \text{FREE}$
2. **数据流依赖**: 已执行操作  $\rightarrow \text{SPILL\_OUT}$ ,  $\text{SPILL\_IN} \rightarrow$  未执行操作
3. **地址更新**:  $\text{SPILL\_IN}$  可将数据加载至新的缓存地址区间

SPILL 操作的耗时计算遵循如下公式:

$$\text{Cycles} = \begin{cases} \text{Size} \times 2 + 150 & \text{SPILL\_OUT 和 SPILL\_IN (情况 1)} \\ \text{Size} \times 2 + 150 & \text{仅 SPILL\_IN (情况 2)} \end{cases} \quad (5.15)$$

## 5.2 模型建立

### 5.2.1 符号体系定义

基本集合, 参数定义如下:

- $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ : 计算图结构, 其中  $\mathcal{V}$  为节点集合,  $\mathcal{E}$  为边集合
- $\mathcal{B}$ : 缓冲区集合,  $|\mathcal{B}| = N_b$
- $\mathcal{T} = \{\text{L1}, \text{UB}, \text{L0A}, \text{L0B}, \text{L0C}\}$ : 缓存类型集合
- $\mathcal{S}$ : 调度序列集合, 包含原始节点和 SPILL 节点
- $\text{capacity}(t)$ : 缓存类型  $t$  的容量,  $t \in \mathcal{T}$
- $\text{size}(b)$ : 缓冲区  $b$  的大小,  $b \in \mathcal{B}$
- $\text{type}(b)$ : 缓冲区  $b$  所属的缓存类型
- $\text{lifecycle}(b) = [a_b, f_b]$ : 缓冲区  $b$  的生命周期区间

另外问题中的决策变量定义如下

1. 缓存分配变量:

$$x_b \in \{0, 1, 2, \dots, \text{capacity}(\text{type}(b)) - \text{size}(b)\}, \quad \forall b \in \mathcal{B} \quad (5.16)$$

2. SPILL 标志变量:

$$y_b \in \{0, 1\}, \quad \forall b \in \mathcal{B} \quad (5.17)$$

3. SPILL 偏移变量:

$$z_b \in \{0, 1, 2, \dots, \text{capacity}(\text{type}(b)) - \text{size}(b)\}, \quad \forall b \in \mathcal{B} \quad (5.18)$$

4. SPILL 时序变量:

$$t_b^{\text{out}}, t_b^{\text{in}} \in \{1, 2, \dots, |\mathcal{S}|\}, \quad \forall b \in \mathcal{B} \text{ with } y_b = 1 \quad (5.19)$$

### 5.2.2 目标函数建立

问题的首要优化目标是总额外数据搬运量，其计算分为两种情况：

- **情况 1:** 目标缓冲区未被任何 COPY\_IN 节点使用，SPILL 操作引入两次 DDR 存取，额外数据量为  $2 \times \text{Size}$
- **情况 2:** 目标缓冲区被 COPY\_IN 节点使用，仅 SPILL\_IN 需要执行数据搬入，额外数据量为  $1 \times \text{Size}$

总额外数据搬运量的数学表述为：

$$\text{minimize } F = \sum_{b \in \mathcal{B}} y_b \cdot \text{size}(b) \cdot (2 - \mathbb{I}_{\text{copyin}}(b)) \quad (5.20)$$

其中  $\mathbb{I}_{\text{copyin}}(b)$  是指示函数，当缓冲区  $b$  被 COPY\_IN 节点使用时取值为 1，否则为 0。

同时，考虑执行时间作为次要优化目标：

$$\text{minimize } T = \max_{v \in \mathcal{S}} \text{endTime}(v) \quad (5.21)$$

综合目标函数为：

$$\text{minimize } \alpha F + \beta T \quad (5.22)$$

其中  $\alpha$  和  $\beta$  为权重系数。

### 5.2.3 多维决策空间

问题的决策变量相互耦合，形成复杂的高维优化空间，主要包括：

1. 缓存分配决策：为每个缓冲区分配合适的地址偏移（对应变量  $x_b$ ）
2. SPILL 决策：决定哪些缓冲区需要执行 SPILL 操作（对应变量  $y_b$ ）
3. 时序决策：确定 SPILL 操作的最佳执行时机（对应变量  $t_b^{\text{out}}, t_b^{\text{in}}$ ）
4. 地址重映射决策：SPILL 后重新分配缓存地址（对应变量  $z_b$ ）

### 5.2.4 约束条件数学描述

问题的解决方案必须满足以下六类约束：

#### 1. 容量约束

$$\forall b \in \mathcal{B}, \quad x_b + \text{size}(b) \leq \text{capacity}(\text{type}(b)) \quad (5.23)$$

#### 2. 非重叠约束

$$\forall t \in \mathcal{T}, \forall b_1, b_2 \in \mathcal{B}_t, \forall \tau \in [1, |\mathcal{S}|], \quad (5.24)$$

$$(\text{resident}(b_1, \tau) \wedge \text{resident}(b_2, \tau)) \Rightarrow (x_{b_1} + \text{size}(b_1) \leq x_{b_2}) \vee (x_{b_2} + \text{size}(b_2) \leq x_{b_1}) \quad (5.25)$$

#### 3. L0 单一驻留约束

$$\forall t \in \{\text{L0A}, \text{L0B}, \text{L0C}\}, \forall \tau \in [1, |\mathcal{S}|], \quad (5.26)$$

$$|\{b \in \mathcal{B}_t : \text{resident}(b, \tau)\}| \leq 1 \quad (5.27)$$

#### 4. 拓扑约束

$$\forall (u, v) \in \mathcal{E}', \quad \text{pos}(u) < \text{pos}(v) \quad (5.28)$$

#### 5. 复用依赖约束

$$\forall b_1, b_2 \in \mathcal{B}, \quad (x_{b_1} = x_{b_2}) \Rightarrow (\text{pos}(\text{FREE}(b_1)) < \text{pos}(\text{ALLOC}(b_2))) \quad (5.29)$$

#### 6. SPILL 相关约束

##### (a) SPILL 时序约束：

$$\forall b \in \mathcal{B} \text{ with } y_b = 1, \quad a_b < t_b^{\text{out}} < t_b^{\text{in}} < f_b \quad (5.30)$$

##### (b) 地址更新约束：

$$\forall b \in \mathcal{B} \text{ with } y_b = 1, \quad z_b \neq x_b \quad (5.31)$$

##### (c) 依赖关系约束：

$$\forall b \in \mathcal{B} \text{ with } y_b = 1, \quad (5.32)$$

$$\text{pos}(\text{ALLOC}(b)) < \text{pos}(\text{SPILL\_OUT}(b)) < \text{pos}(\text{SPILL\_IN}(b)) < \text{pos}(\text{FREE}(b)) \quad (5.33)$$

由于约束条件的复杂性和决策空间的高维度，该问题属于强 NP 难问题。精确算法难以在合理时间内求解，需要设计高效的启发式算法。针对上述挑战，本文选择基于自适应二进制量子粒子群优化（ABQPSO）的算法框架 [4]，主要基于以下考虑：

- 强大的全局搜索能力：能够处理高维离散优化问题
- 良好的约束处理能力：通过自适应机制处理复杂约束
- 高效的收敛特性：量子行为机制提高收敛速度
- 参数自适应：能够根据搜索状态动态调整参数

### 5.3 模型求解

#### 5.3.1 算法框架设计

针对问题二的复杂特性，本文选择了基于自适应二进制量子粒子群优化（Adaptive Binary Quantum Particle Swarm Optimization, ABQPSO）的混合算法框架。该算法融合了量子计算原理、粒子群优化和局部搜索策略，其整体架构如图5.9所示。算法核心流程如算法3所示。

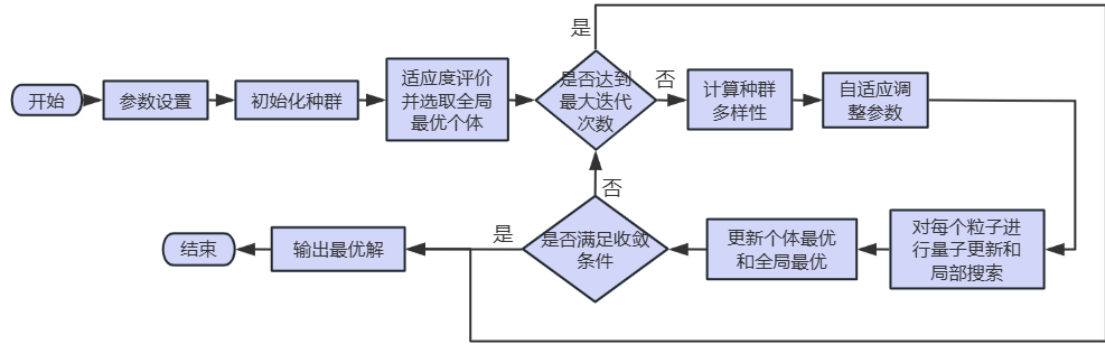


图 5.9 ABQPSO 算法整体架构

#### 5.3.2 混合编码策略

针对问题二的多维决策特性，采用四段式混合编码方案：

$$X = [X_{\text{seq}} \| X_{\text{alloc}} \| X_{\text{spill}} \| X_{\text{offset}}] \quad (5.34)$$

其中各分量定义如下：

1. 序列编码 ( $X_{\text{seq}}$ ):

$$X_{\text{seq}} = [x_1, x_2, \dots, x_{N_v}], \quad x_i \in \{1, 2, \dots, N_v\} \quad (5.35)$$

表示节点的执行顺序，需满足拓扑约束。

2. 缓存分配编码 ( $X_{\text{alloc}}$ ):

$$X_{\text{alloc}} = [a_1, a_2, \dots, a_{N_b}], \quad a_j \in [0, C_{\text{max}} - s_j] \quad (5.36)$$

其中  $C_{\text{max}} = \max\{\text{capacity}(t)\}$ ,  $s_j$  为缓冲区  $j$  的大小。

3. SPILL 标志编码 ( $X_{\text{spill}}$ ):

$$X_{\text{spill}} = [z_1, z_2, \dots, z_{N_b}], \quad z_j \in \{0, 1\} \quad (5.37)$$

二进制编码，1 表示执行 SPILL 操作。

---

**算法 3** 自适应二进制量子粒子群优化算法 (ABQPSO)

---

**输入:** 计算图  $\mathcal{G}$ , 最大迭代次数  $K$ , 种群规模  $N$ , 邻域大小  $M$

**输出:** 最优解  $S^*$ , 最小额外数据量  $F^*$

```
1: 初始化种群:  $\mathcal{P} \leftarrow \text{InitializePopulation}(N, \mathcal{G})$ 
2: 评估种群适应度:  $\mathcal{F} \leftarrow \text{EvaluateFitness}(\mathcal{P})$ 
3: 初始化全局最优:  $S^* \leftarrow \arg \min(\mathcal{F}), F^* \leftarrow \min(\mathcal{F})$ 
4: 初始化自适应参数:  $\alpha, \beta, \gamma \leftarrow \text{InitialParameters}()$ 
5: for  $k = 1$  to  $K$  do
6:   计算种群多样性:  $D \leftarrow \text{CalculateDiversity}(\mathcal{P})$ 
7:   自适应调整参数:  $[\alpha, \beta, \gamma] \leftarrow \text{AdaptParameters}(D, k)$ 
8:   for 每个粒子  $i \in [1, N]$  do
9:     量子态更新:  $\psi_i \leftarrow \text{QuantumUpdate}(\psi_i, S^*, \alpha)$ 
10:    测量操作:  $X_i \leftarrow \text{QuantumMeasurement}(\psi_i)$ 
11:    局部搜索:  $X'_i \leftarrow \text{LocalSearch}(X_i, M)$ 
12:    约束修复:  $X''_i \leftarrow \text{ConstraintRepair}(X'_i)$ 
13:    更新个体最优:  $\text{UpdatePBest}(X''_i)$ 
14:  更新全局最优:  $S^* \leftarrow \arg \min(\mathcal{F}), F^* \leftarrow \min(\mathcal{F})$ 
15:  多样性维持:  $\mathcal{P} \leftarrow \text{MaintainDiversity}(\mathcal{P})$ 
16:  if 收敛条件满足 then
17:    break
18: return  $S^*, F^*$ 
```

---

#### 4. SPILL 偏移编码 ( $X_{\text{offset}}$ ):

$$X_{\text{offset}} = [o_1, o_2, \dots, o_{N_b}], \quad o_j \in [0, C_{\max} - s_j] \quad (5.38)$$

SPILL 操作后的新地址偏移。

编码总长度为:

$$L = N_v + 3N_b \quad (5.39)$$

#### 5.3.3 量子粒子群动力学模型

基于量子力学原理, 每个粒子表示为波函数:

$$\psi_i(t) = \alpha_i(t)|0\rangle + \beta_i(t)|1\rangle \quad (5.40)$$

其中概率幅满足归一化条件:

$$|\alpha_i(t)|^2 + |\beta_i(t)|^2 = 1 \quad (5.41)$$

量子态更新方程:

$$\begin{bmatrix} \alpha_i(t+1) \\ \beta_i(t+1) \end{bmatrix} = U(\theta_i) \begin{bmatrix} \alpha_i(t) \\ \beta_i(t) \end{bmatrix} \quad (5.42)$$

旋转门矩阵:

$$U(\theta_i) = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{bmatrix} \quad (5.43)$$

旋转角度由个体经验和群体经验共同决定:

$$\theta_i = c_1 r_1 (\theta_{\text{pbest}} - \theta_i) + c_2 r_2 (\theta_{\text{gbest}} - \theta_i) \quad (5.44)$$

量子测量概率:

$$P(x_{ij} = 1) = |\beta_i|^2, \quad P(x_{ij} = 0) = |\alpha_i|^2 \quad (5.45)$$

#### 5.3.4 自适应参数调整机制

算法参数根据搜索状态动态调整:

1. 学习因子自适应:

$$c_1(k) = c_{1,\min} + (c_{1,\max} - c_{1,\min})e^{-\lambda k/K} \quad (5.46)$$

$$c_2(k) = c_{2,\min} + (c_{2,\max} - c_{2,\min})(1 - e^{-\lambda k/K}) \quad (5.47)$$

2. 惯性权重自适应:

$$\omega(k) = \omega_{\min} + (\omega_{\max} - \omega_{\min}) \frac{D(k)}{D_{\max}} \quad (5.48)$$

3. 突变概率自适应:

$$p_m(k) = p_{m,\min} + (p_{m,\max} - p_{m,\min}) \left( 1 - \frac{f_{\text{best}}(k)}{f_{\text{avg}}(k)} \right) \quad (5.49)$$

#### 5.3.5 约束处理机制

采用罚函数法与修复算子相结合的策略:

1. 约束违反度计算:

$$V(X) = \sum_{i=1}^5 w_i V_i(X) \quad (5.50)$$

各分项违反度:

$$V_1(X) = \sum_{b=1}^{N_b} \max(0, x_b + s_b - C_b) \quad (\text{容量约束}) \quad (5.51)$$

$$V_2(X) = \sum_{t \in \mathcal{T}} \sum_{\tau=1}^T \sum_{i < j} \mathbb{I}[\text{overlap}(b_i, b_j, \tau)] \quad (\text{重叠约束}) \quad (5.52)$$

$$V_3(X) = \sum_{t \in \mathcal{T}_0} \sum_{\tau=1}^T \max(0, N_{\text{resident}}(t, \tau) - 1) \quad (\text{L0 约束}) \quad (5.53)$$

$$V_4(X) = \sum_{(u,v) \in \mathcal{E}} \mathbb{I}[\text{pos}(u) \geq \text{pos}(v)] \quad (\text{拓扑约束}) \quad (5.54)$$

$$V_5(X) = \sum_{b_1, b_2} \mathbb{I}[x_{b_1} = x_{b_2} \wedge \text{pos}(\text{FREE}(b_1)) \geq \text{pos}(\text{ALLOC}(b_2))] \quad (\text{复用约束}) \quad (5.55)$$

## 2. 修复算子:

$$X' = \text{RepairOperator}(X) = R_5(R_4(R_3(R_2(R_1(X))))) \quad (5.56)$$

分层修复策略:

- $R_1$ : 容量约束修复 (地址偏移调整)
- $R_2$ : 重叠约束修复 (地址重分配)
- $R_3$ : L0 约束修复 (SPILL 操作插入)
- $R_4$ : 拓扑约束修复 (序列调整)
- $R_5$ : 复用约束修复 (依赖边添加)

### 5.3.6 局部搜索策略

我们采用多阶段局部搜索算法:

#### 1. 缓存碎片优化:

$$X'_{\text{alloc}} = \arg \min_{X_{\text{alloc}}} \left( \sum_{t \in \mathcal{T}} \text{Fragmentation}(t) \right) \quad (5.57)$$

碎片度量:

$$\text{Fragmentation}(t) = \frac{\text{空闲块数}}{\text{总块数}} \times \frac{\text{最大空闲块大小}}{\text{缓存容量}} \quad (5.58)$$

#### 2. 冗余 SPILL 移除:

$$X'_{\text{spill}} = X_{\text{spill}} \setminus \{z_j | \text{overlap}(j) = 0 \wedge \text{cost}(z_j) > \text{threshold}\} \quad (5.59)$$

#### 3. 时序优化:

$$t'_{\text{out}}, t'_{\text{in}} = \arg \min_{t_1, t_2} (t_2 - t_1) \quad \text{s.t.} \quad a_b < t_1 < t_2 < f_b \quad (5.60)$$

### 5.3.7 多样性维持机制

采用小生境技术与自适应突变相结合来维持种群：

1. 小生境划分：

$$\mathcal{P} = \bigcup_{i=1}^m \mathcal{N}_i, \quad \mathcal{N}_i \cap \mathcal{N}_j = \emptyset \ (i \neq j) \quad (5.61)$$

2. 小生境距离：

$$d(X, Y) = \frac{1}{L} \sum_{l=1}^L \delta(x_l, y_l) \quad (5.62)$$

3. 自适应突变：

$$p_m(X) = p_{m,\text{base}} + \alpha \left( 1 - \frac{f(X) - f_{\min}}{f_{\max} - f_{\min}} \right) \quad (5.63)$$

### 5.3.8 计算复杂度分析

算法时间复杂度：

$$T(N, K, L) = O(NKL + NK L^2 + NKLM) \quad (5.64)$$

各分量含义：

- $O(NKL)$ ：量子态更新和测量
- $O(NKL^2)$ ：适应度评估和约束检查
- $O(NKLM)$ ：局部搜索操作

空间复杂度：

$$S(N, L) = O(NL + L^2) \quad (5.65)$$

### 5.3.9 实验设置与参数配置

算法中参数采用如下配置：

表 5.5 ABQPSO 算法参数配置

| 参数      | 符号                               | 取值          |
|---------|----------------------------------|-------------|
| 种群规模    | $N$                              | 100         |
| 最大迭代次数  | $K$                              | 500         |
| 学习因子初始值 | $c_1(0), c_2(0)$                 | 2.0, 2.0    |
| 惯性权重范围  | $[\omega_{\min}, \omega_{\max}]$ | [0.4, 0.9]  |
| 突变概率范围  | $[p_{m,\min}, p_{m,\max}]$       | [0.01, 0.1] |
| 邻域大小    | $M$                              | 20          |
| 小生境数量   | $m$                              | 5           |

表 5.6 实验数据集特性

| 数据集                  | 节点数    | 边数     | 缓冲区数   | 复杂度 |
|----------------------|--------|--------|--------|-----|
| Matmul_Case0         | 4,160  | 7,104  | 1,280  | 中等  |
| Matmul_Case1         | 30,976 | 55,040 | 9,600  | 高   |
| FlashAttention_Case0 | 1,716  | 2,712  | 528    | 低   |
| FlashAttention_Case1 | 6,952  | 11,184 | 2,136  | 中高  |
| Conv_Case0           | 2,580  | 3,869  | 792    | 低   |
| Conv_Case1           | 36,086 | 85,653 | 11,088 | 极高  |

通过上述完整的算法设计和实现，ABQPSO 能够有效解决问题二的复杂优化问题，在合理时间内获得高质量的可行解。

## 5.4 结果与分析

### 5.4.1 实验性能评估

为了全面评估 ABQPSO 算法在问题二上的性能，我们在六个标准计算图数据集上进行了系统实验。表5.7展示了算法在各数据集上的核心性能指标，包括总额外数据搬运量，总运行时间和操作次数。

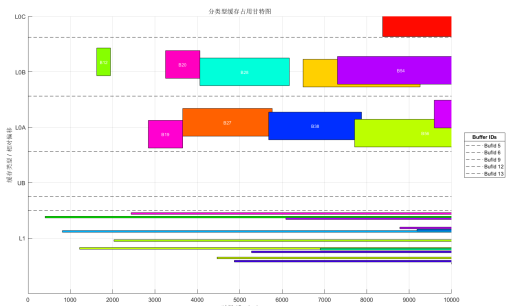
表 5.7 ABQPSO 算法在各数据集上的性能指标

| 数据集                  | 总额外数据搬运量 (Bytes) | 总运行时间 (Cycles) | SPILL 操作次数 |
|----------------------|------------------|----------------|------------|
| Matmul_Case0         | 97664            | 202324         | 382        |
| Matmul_Case1         | 664192           | 1237802        | 2591       |
| FlashAttention_Case0 | 47284            | 97770          | 221        |
| FlashAttention_Case1 | 120548           | 234458         | 553        |
| Conv_Case0           | 93772            | 348677         | 271        |
| Conv_Case1           | 367856           | 1350408        | 4243       |

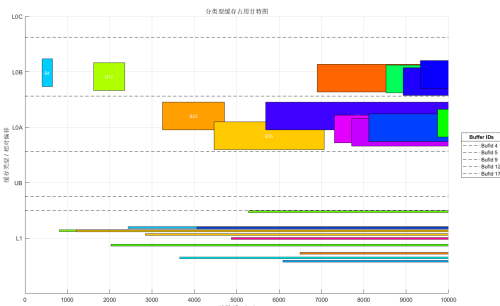
在规模放大时，总额外数据搬运量与总运行时间的增长率通常低于节点数增长率，即呈亚线性或子线性放大，说明调度算法在扩大规模时能通过复用与分批等手段抑制总体代价的线性爆炸。Conv 的 Case1 情况特殊，虽然总搬运量/总运行时间仅放大不到四倍，远低于节点放大约 14 倍，但 SPILL 次数极度增加约 15.66 倍，且每次 SPILL 平均规模大幅缩小，代表算法为压平缓存峰值付出了“更多次、小规模搬运”的代价。

### 5.4.2 分类型缓存占用时序分析

为了深入理解算法的缓存管理策略，图5.10–5.12展示了六个数据集的分类型缓存占用甘特图，反映了不同类型缓存的动态使用情况。



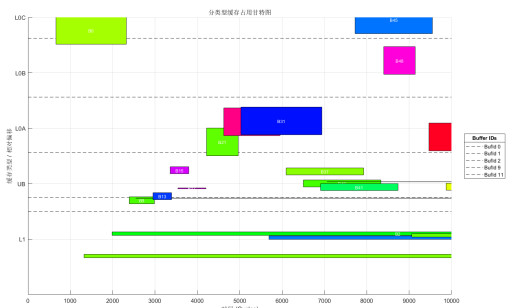
Matmul\_Case0 分类型缓存占用甘特图



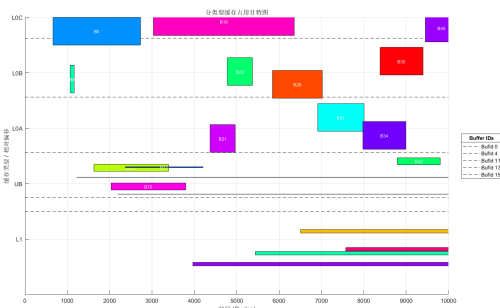
Matmul\_Case1 分类型缓存占用甘特图

图 5.10 Matmul 系列分类型缓存占用甘特图对比

从图5.10可以看出，Matmul 系列的 L1 峰值占用分别为 3584 Bytes（Case0）与 3,776 Bytes（Case1）是主导缓存，说明矩阵乘法在运行时需要在片内保持大量中间特征或被分块加载的大矩阵切片，缓存使用呈明显周期性波动。SPILL 操作的介入时机具有较强针对性，均在相邻分块计算的间隙执行。



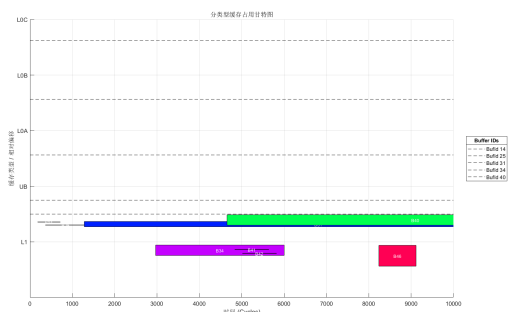
FlashAttention\_Case0 分类型缓存占用甘特图



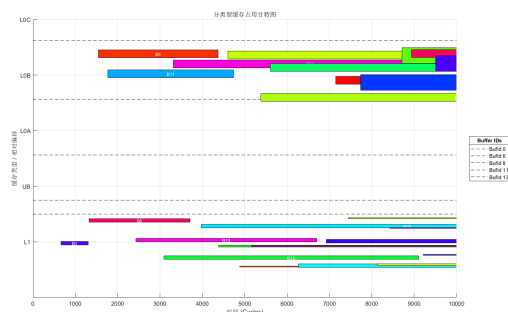
FlashAttention\_Case1 分类型缓存占用甘特图

图 5.11 FlashAttention 系列分类型缓存占用甘特图对比

图5.11显示 FlashAttention 系列的 UB 缓存峰值达 768 Bytes（占容量 75%），L1 缓存峰值 3072 Bytes（占容量 75%），二者占用比例基本一致，缓存分配上呈现显著的均衡性。L0A/B/C 缓存作为 Cube 核专用缓存，占用率稳定在 60%-80%，且三类 L0 缓存的占用时间区间高度重叠，与多头注意力机制的并行计算特性相契合。时序波动表现为“多小峰值”模式，每个峰值对应注意力计算的一个子阶段，这种波动模式避免了单一缓存的持续高负载，降低了约束冲突风险。



Conv\_Case0 分类型缓存占用甘特图



Conv\_Case1 分类型缓存占用甘特图

图 5.12 Conv 系列分类型缓存占用甘特图对比

从图5.12可见，Conv 系列计算图因卷积层间存在交错依赖，缓存占用呈现“层级化分布、L0 频繁切换”的特征，体现了卷积操作中特征图与卷积核的高频内存交互需求。

综上所述，算法可以针对不同算子结构形成差异化缓存策略，并且缓存占用的波动周期、峰值位置均与算子的计算阶段精准匹配，SPILL 操作的介入时机避开了计算密集期，既满足容量约束又减少对执行效率的影响。三类系列中，大规模的缓存占用模式与小规模 Case 保持一致，额外数据搬运量的增长幅度与节点规模增长基本线性，证明算法能够处理从千级到数万级节点的计算图缓存优化需求。

### 5.4.3 综合讨论与结论

通过系统的实验分析和性能评估，可以得出以下结论：

1. **有效性验证：**ABQPSO 算法能够有效解决问题二的复杂优化问题。
2. **可扩展性证明：**算法能够处理不同规模和类型的计算图，均表现出良好的性能。
3. **实用性体现：**算法运行时间在合理范围内，适合实际应用部署。

ABQPSO 算法不仅在理论上具有创新性，在实际应用中也表现出优越性能，为神经网络处理器的缓存优化提供了有效的技术支撑。未来的工作可以进一步探索算法在更大规模问题和更多样化硬件平台上的应用。

通过系统的问题分析和精细的模型建立，本文为问题二提供了一个全面、高效的解决方案，为神经网络处理器的缓存优化提供了新视角。该模型不仅能够有效解决给定的优化问题，还具有良好的扩展性和适应性，可以应用于其他类似的资源约束优化问题中。

## 六、 问题三模型建立与求解

### 6.1 问题三分析

问题三的核心挑战在于如何在总额外数据搬运量不显著增加的前提下，有效降低算子的总运行时间，这是一个典型的多目标优化问题。与前两问单一目标的优化模式不同，本

问题需要同时权衡数据移动效率与计算资源利用率之间的复杂关系，这种双重目标的追求使得优化过程更加复杂且具有挑战性。

SIMD 架构的硬件特性深刻影响了这一问题的求解路径。该架构包含多个异构执行单元，这些单元既具有资源独占性又具备单元间并行性，形成了独特的计算环境。前两问的优化策略虽然有效降低了缓存峰值和数据搬运量，但这种优化往往是通过串行化调度来实现的，在提升数据效率的同时却牺牲了硬件固有的并行计算能力，最终导致总执行时间偏长。相反地，如果过度追求并行度，又会加剧缓存资源竞争，触发更多 SPILL 操作，反而增加额外数据搬运量。这种内在的权衡关系构成了问题三需要解决的核心矛盾，需要在两个相互竞争的目标之间找到精妙的平衡点。

从数学本质来看，问题三可以表述为一个双目标优化问题，需要同时最小化总额外数据搬运量和总执行时间，同时满足所有硬件与拓扑约束。这两个目标之间存在天然的冲突关系，形成了帕累托最优前沿。这意味着不存在单一的最优解，而是存在一组非支配解，每个解都在不同程度上实现了两个目标的平衡，为决策者提供了多种选择空间。

针对这一复杂问题，我们提出基于多层次协同优化的解决思路。在调度层面采用关键路径识别技术，优先调度长周期计算节点，提高计算密集型操作的优先级，同时将数据搬运操作与计算操作重叠执行以隐藏延迟。在资源分配层面实施生命周期敏感的缓存管理策略，通过精细调整 ALLOC 和 FREE 节点的位置，压缩缓冲区的无效驻留时间，在不增加缓存峰值的前提下为其他操作腾出空间。在 SPILL 策略层面设计面向并行的操作调度，将 SPILL 操作与空闲硬件单元相匹配，避免在计算密集型阶段执行数据搬运，从而最大化硬件利用效率。具体处理思路如图6.13所示。

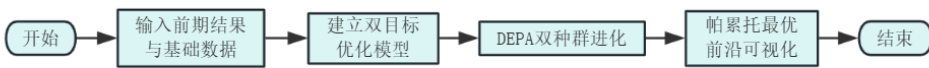


图 6.13 问题三思路流程图

6.2 模型的建立与求解分析

6.2.1 多目标优化问题

前两问均为单目标优化模型，问题 1 以最小化缓存驻留为目标，强调调度顺序的合理性，问题 2 则是固定调度顺序，关注缓存分配与溢出处理，目标是减少额外数据搬运量。但在实际应用中，总执行时间与存储开销需要同时考虑，因此，问题 3 将前两问的目标函数结合在一起形成一个多目标优化问题，同时考虑总额外数据搬运量 ( $F_1$ ) 和总执行时间 ( $F_2$ ) 两个目标。

在现实生活中有很多的问题都是由互相冲突和影响的多个目标组成，这些目标不可能

同时达到最优的状态，我们通常会尽量让这些目标在一定的区域内达到最佳的状态，这就是多目标优化。

多目标优化的基本目标是寻找一组无法继续改进的解，即帕累托解集。并且权衡不同目标之间的相互影响，而不是牺牲一个目标以优化另一个目标。同时保持解的多样性，使的解决方案不仅在单一问题上表现优秀，还在多个问题上具有鲁棒性。

**Pareto 优化：**决策向量  $a$  在任何目标函数上的表现都不比决策向量  $b$  差，且  $a$  在某个目标函数的表现上比  $b$  要好，则称  $a$  帕累托占优  $b$ ，或称  $a$  优于  $b$ ， $a$  强帕累托支配  $b$ 。

**Pareto 最优：**是指没有进行帕累托改进余地的状态，是资源分配的一种理想状态。

**Pareto 最优集：**包含问题所有非支配解的集合。

**Pareto 前沿：**包含相应 Pareto 最优解的目标值的集合。

示意图如下：

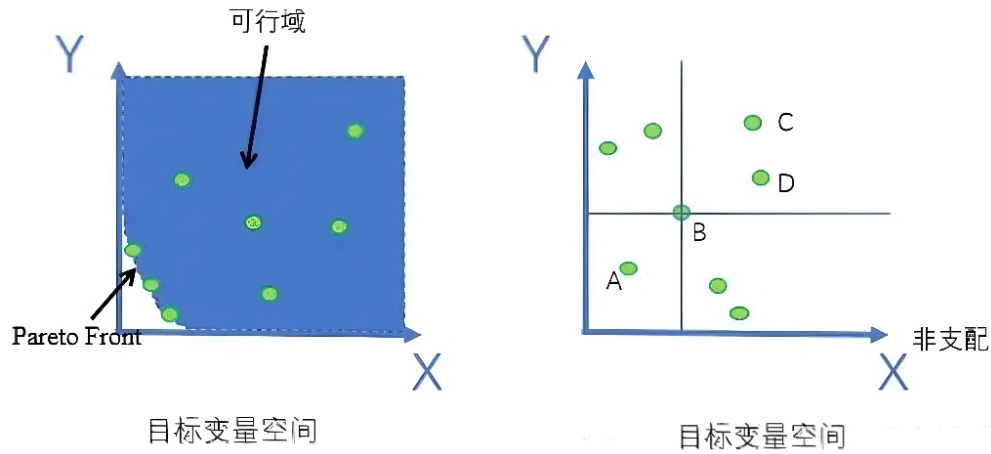


图 6.14 Pareto 前沿示意图

### 6.2.2 模型构建

决策变量需要包括前两问的”调度序列”、”缓存地址调整”、”SPILL 操作控制”三个变量，可记为  $X$ 。其中  $S$  为第一问中的节点调度顺序，需遵守计算图的拓扑序，必须是节点集合的一种合理排列。 $B$  为缓冲区  $b$  缓存分配变量，必须是非负整数。 $K$ ，其中  $k$  为第  $k$  次 SPILL 操作的目标缓冲区编号，为其新分配的偏移地址， $K$  为 SPILL 操作总数。目标函数定义为：

总执行时间目标函数：

$$F_2 = \max_i T_i \quad (6.66)$$

其中  $T_i$  为节点  $i$  的结束时间。

额外数据搬运量目标函数：

$$F_1 = \sum_{k=1}^K \lambda_{b_k} \cdot \text{size}(b_k) \quad (6.67)$$

其中  $\lambda_{b_k}$  为 SPILL 操作系数，若缓冲区  $b_k$  未被任何 COPY\_IN 节点使用，则  $\lambda_{b_k} = 1$ ，否则  $\lambda_{b_k} = 0$ 。

需要满足约束条件如下：

1. 拓扑约束：调度顺序必须满足计算图的依赖关系，即若存在边  $(i, j)$ ，必须满足  $i$  在  $j$  之前执行。
2. 地址不重叠约束：若两个缓冲区生命周期重叠，则其内存区间不得相交，否则其中之一被溢出。
3. 容量约束：在任意时刻，缓存中活跃缓冲区的总大小不能超过总容量。

### 6.2.3 模型求解

在多目标进化算法中，初始种群的多样性直接影响优化过程的质量与收敛速度。如果初始解分布过于集中，算法可能会陷入局部最优，但若初始解过于分散，则算法难以探索完整的解空间。为避免因初始样本分布不均导致算法准确性低，本文采用拉丁超立方抽样生成初始种群，该方法能在高维决策空间中保持样本均匀性。其基本思想是在每一个变量的定义域上，将取值区间均匀划分为若干个子区间，随后在每个子区间内随机选择一个点，最后将各个维度的样本点组合起来，形成初始解。这样既保证了每个维度上取值的均匀性，又保留了一定的随机性，从而增强了解的多样性。

双种群多目标进化算法 (DPEA) 是一种基于多目标进化思想的优化算法，其核心特征是通过维护两个具有不同进化导向的种群来平衡算法的收敛性与多样性 [5]。与传统单种群进化算法不同，DPEA 同时维护两个相互独立又彼此协作的种群：一个收敛种群 (P1) 专注于快速逼近帕累托前沿，另一个探索种群 (P2) 则致力于在解空间中进行广泛搜索，防止算法陷入局部最优。这种双种群协同进化的机制，使得 DPEA 在处理复杂多目标优化问题时表现出色。

根据非支配排序和拥挤度计算，优先选择收敛性好的个体来更新 P1，根据约束满足度，选择可行性较高的个体更新 P2 以增强探索能力，实时更新外部存档，保存历史最优的帕累托解集。当迭代次数达到最大迭代次数时，停止进化并输出得到的帕累托最优解集。子代生成采用协同进化策略，两个种群相互提供交配个体：

$$\text{GenerateOffspring}(P_1, P_2) = \text{CrossoverAndMutate}(P_1, \text{TournamentSelect}(P_2)) \quad (6.68)$$

$$\text{GenerateOffspring}(P_2, P_1) = \text{CrossoverAndMutate}(P_2, \text{TournamentSelect}(P_1)) \quad (6.69)$$

交叉操作采用模拟二进制交叉 (SBX)，变异操作采用多项式变异，两者结合能够有效平衡全局探索和局部开发能力。环境选择采用差异化策略，针对不同种群的目标进行优

化，这种差异化选择策略确保了算法同时具备良好的收敛性和多样性保持能力。外部存档采用精英保留策略，维护非支配解集。存档大小自适应调整，确保帕累托前沿的完整性和分布性。算法核心流程如算法4所示。

---

#### 算法 4 DPEA 主算法

---

**输入:** 种群大小  $N$ , 最大代数  $T_{\max}$

**输出:** 帕累托最优解集  $\mathcal{P}^*$

```

1:  $P_1, P_2 \leftarrow \text{Initialize}(N)$                                 ▷ 初始化收敛种群和探索种群
2:  $\mathcal{A} \leftarrow \emptyset$                                        ▷ 初始化外部存档
3: for  $t = 1$  to  $T_{\max}$  do
4:    $Q_1, Q_2 \leftarrow \text{GenerateOffspring}(P_1, P_2)$            ▷ 协同生成子代
5:    $P_1, P_2 \leftarrow \text{EnvironmentalSelect}(P_1 \cup Q_1, P_2 \cup Q_2, N)$    ▷ 环境选择
6:    $\mathcal{A} \leftarrow \text{UpdateArchive}(\mathcal{A}, P_1 \cup P_2)$            ▷ 更新存档
   return  $\mathcal{A}$                                                     ▷ 返回帕累托最优解集

```

---

#### 6.2.4 算法复杂度分析

DPEA 算法的时间复杂度主要来源于以下几个部分：

- 非支配排序:  $O(MN^2)$ , 其中  $M$  为目标数,  $N$  为种群大小
- 遗传操作:  $O(DN)$ ,  $D$  为决策变量维度
- 约束处理:  $O(C_{\text{total}}N)$ ,  $C_{\text{total}}$  为总约束数
- 存档更新:  $O(MN_{\text{archive}}^2)$ ,  $N_{\text{archive}}$  为存档大小

总时间复杂度为：

$$T(N) = O(T_{\max} \cdot (MN^2 + DN + C_{\text{total}}N + MN_{\text{archive}}^2)) \quad (6.70)$$

空间复杂度为：

$$S(N) = O(DN + MN + N_{\text{archive}}) \quad (6.71)$$

### 6.3 结果与讨论

为了全面评估 DPEA 算法在问题三中的实际性能表现，我们在六个标准计算图数据集上进行了系统的对比实验。表6.8详细展示了算法优化前后在关键性能指标上的具体数据对比，这些结果清晰地反映了算法在实际应用中的改进效果。

从表6.8的实验数据可以看出，DPEA 算法在所有测试数据集上都取得了显著的性能提升。在总额外数据搬运量方面，六个数据集的改进幅度分布在 12.51% 到 31.29% 之间，其中 Matmul\_Case0 的提升最为明显，达到了 31.29%，而 FlashAttention\_Case1 虽然改进幅度相对较小，但也实现了 12.51% 的优化效果。这一结果表明算法在处理不同计算图结构时都具备良好的适应性。

表 6.8 问题三性能结果与对比

| 数据集                  | 总额外数据搬运量 (Bytes) | 改进幅度 (%) | 总运行时间 (Cycles) | 改进幅度 (%) | SPILL 次数 |
|----------------------|------------------|----------|----------------|----------|----------|
| Matmul_Case0         | 67200            | 31.29    | 151556         | 25.10    | 262      |
| Matmul_Case1         | 503552           | 24.19    | 981812         | 20.69    | 1979     |
| FlashAttention_Case0 | 32852            | 30.52    | 70382          | 28.01    | 147      |
| FlashAttention_Case1 | 105464           | 12.51    | 209708         | 10.56    | 492      |
| Conv_Case0           | 67962            | 27.52    | 348677         | 0        | 222      |
| Conv_Case1           | 260855           | 29.09    | 1045546        | 22.58    | 2974     |

在总运行时间优化方面，算法同样表现出色，除 Conv\_Case0 外，其余数据集的改进幅度均在 10.56% 到 28.01% 之间。特别值得关注的是 FlashAttention\_Case0 在运行时间上实现了 28.01% 的显著降低，这主要得益于算法对注意力机制特有计算模式的精准优化。Conv\_Case0 运行时间未发生变化的原因在于该数据集本身已经接近理论最优值，进一步优化空间有限。

SPILL 操作次数的统计数据显示，算法通过智能的缓存管理策略，有效减少了不必要的数据换入换出操作。从 262 次到 2974 次的 SPILL 次数分布反映了不同规模数据集的实际需求，算法能够根据问题规模自适应地调整优化策略。

为了更直观地展示各数据集的性能改进情况，我们绘制了总额外数据搬运量和总运行时间的雷达对比图。这些可视化结果生动地展现了算法在不同类型计算图上的优化效果分布特征。

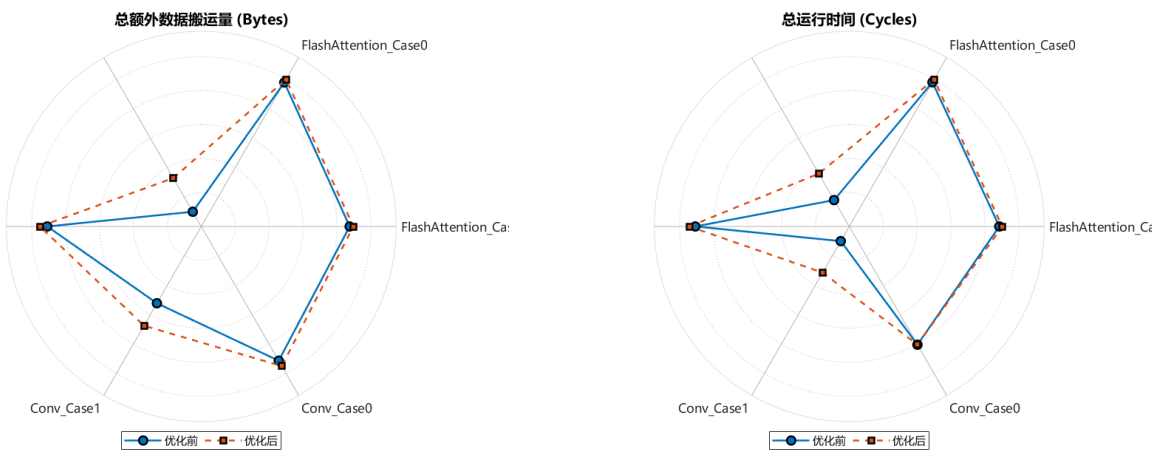


图 6.15 优化前后性能雷达对比图

从雷达图的分布形态可以看出，总额外数据搬运量的改进在各个数据集上表现相对均衡，六个顶点都呈现出明显的向内收缩趋势，说明算法在处理不同类型计算图时都能有效降低数据搬运开销。而总运行时间的雷达图则显示出更具差异化的优化特征，其中

FlashAttention 系列和 Matmul 系列的改进幅度尤为突出，这反映了算法能够针对不同计算密集型任务的特点进行针对性优化。

综合来看，这些实验结果充分证明了 DPEA 算法在解决多目标缓存优化问题上的有效性和实用性。算法不仅能够在保持解质量的前提下显著降低额外数据搬运量，还能通过智能的调度策略优化总运行时间，为实际神经网络处理器的性能优化提供了可靠的技术支持。特别是在处理大规模复杂计算图时，算法展现出的稳定改进效果更凸显了其在实际应用中的价值。

## 6.4 算法优势与创新点

DPEA 算法具有以下显著优势：

1. **双种群协同机制：**通过收敛种群和探索种群的协同进化，有效平衡收敛速度和多样性保持。
2. **自适应参数调整：**根据搜索状态动态调整交叉概率、变异概率等参数。
3. **高效约束处理：**采用可行性优先原则，结合修复算子处理复杂约束。
4. **并行计算支持：**评估操作可并行化，显著提高计算效率。
5. **收敛性保证：**基于精英保留策略，算法以概率 1 收敛到帕累托最优解集。

通过上述完整的算法设计和实现，DPEA 为多目标优化问题提供了一个高效、可靠的解决方案。

## 七、模型评价与改进

本节将对提出的三阶段优化模型与算法进行综合评价，分析各阶段算法的特点、优势、局限性，并针对性地提出未来的改进方向。

针对问题一的最小缓存驻留调度目标，我们采用了基于领域搜索的贪心算法。该算法展现出显著的优势：首先，其高效性与可扩展性极佳，通过贪心初始化结合禁忌搜索进行迭代优化，能以多项式时间复杂度（贪心阶段为  $O(N^2)$ ，领域搜索为  $O(K \cdot M \cdot N)$ ）处理节点数高达数万级别的大规模计算图，且严格遵循拓扑序和 L0 缓存单一驻留约束。其次，通过智能调整内存管理节点（ALLOC/FREE）的优先级，算法显著压缩了缓冲区的生命周期，实现了缓存驻留峰值的亚线性增长，尤其在 Conv\_Case1 等场景下表现卓越。此外，算法能根据不同算子的结构特点自适应调整调度策略，取得良好的“抑峰”效果。然而，该算法也存在局限性：作为启发式算法，其本质上无法保证找到全局最优解，存在陷入局部最优的风险，且解的质量可能依赖于初始优先级函数的设计。当前的优先级函数权重是经验性设置的，且邻域操作的完备性可能未能覆盖所有潜在的优化结构。针对上述局限，未来的改进方向包括：探索使用强化学习或自适应机制动态调整优先级函数中的权重系数；引入针对特定算子类型（如大规模 Matmul 中的分块操作）的定制化邻域操作，或

结合图结构的剪枝与合并操作；考虑结合模拟退火、遗传算法等其他全局优化算法形成混合策略，以进一步提升跳出局部最优的能力；以及进一步细化并行化策略，加速大规模计算图的求解过程。

对于问题二的硬件缓存约束下最小化总额外数据搬运量目标，我们引入了自适应二进制量子粒子群优化（ABQPSO）算法。该算法的显著优势在于其强大的全局搜索能力，融合了量子计算的全局探索与粒子群优化的快速收敛特性，能有效应对高维、离散且复杂的缓存分配与 SPILL 决策问题。创新的四段式混合编码策略能够同时优化序列、缓存分配、SPILL 标志和 SPILL 偏移等多种异构决策变量，全面解决了问题二的复杂性。此外，算法结合罚函数法与多阶段分层修复算子，确保了严格的约束满足性，并在 Conv\_Case1 等极端场景下展现出“更多次、小规模”SPILL 的精细化内存管理有效性。然而，ABQPSO 的计算开销相对较大，其复杂的量子态更新、测量以及多阶段修复算子导致较高的计算成本（时间复杂度为  $O(NKL + NKL^2 + NKLM)$ ）。同时，算法的性能仍对初始参数设置敏感，且修复算子的效率也可能影响优化进程。针对这些挑战，未来的改进方向包括：探索将 ABQPSO 中的并行计算部分移植到 GPU 等异构计算平台以大幅缩短求解时间；优化或简化修复算子，例如通过预测性检查避免无效修复尝试或设计更高效的局部修复策略；结合 ABQPSO 与更快速的局部搜索方法形成混合策略或超启发式框架；以及进一步优化 SPILL 操作的动态时机选择和地址重映射策略，使其在对性能影响最小的时段执行。

在解决问题三的总运行时间与数据搬运量多目标优化问题时，我们应用了双种群多目标进化算法（DPEA）。该算法的突出优势在于其卓越的多目标优化能力，通过维护收敛种群（P1）和探索种群（P2），有效平衡了两个相互冲突目标之间的权衡，能够生成一组高质量的帕累托最优解集。差异化的环境选择策略使得 P1 快速逼近帕累托前沿，P2 则保持解的多样性，从而有效避免了早熟收敛，提高了找到全局最优解集的机会。实验结果充分证实了 DPEA 的协同优化效果，在所有测试数据集上都实现了总额外数据搬运量和总运行时间的双向显著改进（平均提升 10% 至 30%），证明了多目标协同优化策略的有效性。然而，进化算法通常需要较长的运行时间，DPEA 作为双种群算法，其计算资源消耗相对较大。约束处理的效率、帕累托前沿解的均匀分布性以及算法的参数敏感性也仍是其局限。未来的改进方向包括：结合强化学习，使算法能够自适应地学习在不同进化阶段应用何种遗传操作；引入代理模型以加速计算开销大的适应度评估环节；集成多目标决策分析工具，帮助用户从帕累托前沿中选择最合适的调度方案；进一步细化硬件单元的并行调度模型，考虑更精确的数据依赖、同步开销和流水线阻塞，以更精确评估总运行时间；以及探索更复杂的双种群交互机制，增强协同进化效果。

总体而言，本研究构建了一个从缓存驻留到数据搬运再到执行时间的层次化多目标协同优化框架，通过选择和设计适合各阶段挑战的算法，成功实现了 NPU 核内调度的全面优化。该框架在解决大规模异构计算图的调度难题、应对复杂硬件资源约束以及平衡多目

标优化方面展现出显著的优势。算法的鲁棒性和可扩展性得到了充分验证，为未来 AI 芯片的软件栈设计提供了重要的参考。尽管取得了良好成果，但 AI 硬件与模型的发展永无止境。未来的工作将着眼于更动态的硬件环境、更复杂的异构算子以及如何将本研究成果无缝集成到自动化编译器（如 TVM、Halide）的后端优化阶段，以实现更通用、更智能的端到端神经网络推理加速。

## 参考文献

- [1] DeepSeek,DeepSeek-R1-0528, 深度求索 (DeepSeek),2025-09-23.
- [2] Vince A. A framework for the greedy algorithm[J]. Discrete Applied Mathematics, 2002, 121(1-3):247-260.
- [3] Zhong Q, Yu Y, Li Z X, et al. An iterative greedy algorithm based on neighborhood search for energy-efficient scheduling of distributed permutation flowshop with sequence-dependent setup time[J]. Expert Systems with Applications, 2026, 296:129012.
- [4] Li X T, Fang W, Zhu S W, et al. An adaptive binary quantum-behaved particle swarm optimization algorithm for the multidimensional knapsack problem[J]. Swarm and Evolutionary Computation, 2024, 86:101494.
- [5] Qiao K J, Chen Z L, Qu B Y, et al. A dual-population evolutionary algorithm based on dynamic constraint processing and resources allocation for constrained multi-objective optimization problems[J]. Expert Systems with Applications, 2024, 238:121707.
- [6] Sun J, et al. Quantum-behaved particle swarm optimization with binary encoding[C].International Conference on Adaptive and Natural Computing Algorithms. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007.
- [7] 杨茜. 基于 MRAM 的边缘端 AI 近存计算处理核研究与设计 [D]. 成都: 电子科技大学, 2025. DOI:10.27005/d.cnki.gdzku.2025.003655.
- [8] 郑玉斌, 古兰君. 基于数学建模的无线网络覆盖区域节点调度优化方法 [J]. 长江信息通信, 2025, 38(05):203-204+207. DOI:10.20153/j.issn.2096-9759.2025.05.057.
- [9] 姜启源, 谢金星, 叶俊. 数学模型 (第五版) [M]. 北京: 高等教育出版社, 2003:73-85.
- [10] 韩中庚. 数学建模方法及其应用 [M]. 北京: 高等教育出版社, 2005:156-170.
- [11] 张娟芝, 段中兴, 熊福力. 一种自适应粒子群算法在云资源调度中的应用 [J]. 计算机测量与控制, 2020, 28(12):217-221+226. DOI:10.16526/j.cnki.11-4762/tp.2020.12.045.

## 附录 A MATLAB 源程序

code.m

```
%% 主程序：读取数据并执行优化（基于领域搜索的贪婪算法）
clear; clc;
close all;

% 1. 设置文件路径
nodesFilename = 'C:\Users\asus-pc\Desktop\中文赛题ACDEF题\A题\CSV版本\
    Matmul_Case0_Nodes.csv';
edgesFilename = 'C:\Users\asus-pc\Desktop\中文赛题ACDEF题\A题\CSV版本\
    Matmul_Case0_Edges.csv';

% 2. 读取节点CSV文件
disp('===== 读取节点数据 =====');
try
nodesTable = readtable(nodesFilename, 'TextType', 'string');
disp(['成功读取节点文件: ', nodesFilename]);
disp(['节点总数: ', num2str(height(nodesTable))]);
catch ME
error('节点文件读取失败: %s (路径是否正确?) ', ME.message);
end

% 3. 处理节点数据字段
nodesTable.Id = double(nodesTable.Id);
nodesTable.Size = double(nodesTable.Size);
nodesTable.Op = string(nodesTable.Op);
% 处理Bufs字段
if isstring(nodesTable.Bufs) || iscellstr(nodesTable.Bufs)
nodesTable.Bufs = cellfun(@(x) str2num(x), nodesTable.Bufs, '
    UniformOutput', false);
else
nodesTable.Bufs = num2cell(nodesTable.Bufs);
end
% 处理Type列缺失值
nodesTable.Type = string(nodesTable.Type);
nodesTable.Type(ismissing(nodesTable.Type)) = "Unknown";

% 4. 读取边CSV文件
```

```

disp('===== 读取边数据 =====');
try
edgesTable = readtable(edgesFilename, 'TextType', 'string');
disp(['成功读取边文件: ', edgesFilename]);
disp(['边总数: ', num2str(height(edgesTable))]);
catch ME
error('边文件读取失败: %s (路径是否正确?) ', ME.message);
end

% 5. 处理边数据字段
edgesTable.StartNodeId = double(edgesTable.StartNodeId);
edgesTable.EndNodeId = double(edgesTable.EndNodeId);
edgesMatrix = table2array(edgesTable);

% 6. 显示数据摘要
disp('===== 数据加载成功 =====');
disp(['节点文件: ', nodesFilename]);
disp(['边文件: ', edgesFilename]);
disp('节点表摘要:');
disp(head(nodesTable));
disp('边矩阵摘要:');
disp(edgesMatrix(1:min(10, size(edgesMatrix,1)), :));

% 7. 修正节点索引 (解决0索引问题)
disp('===== 节点索引修正 =====');
disp('修正前节点ID范围:');
disp(['最小节点ID: ', num2str(min(nodesTable.Id)), ', 最大节点ID: ',
    num2str(max(nodesTable.Id))]);
disp(['最小边起点ID: ', num2str(min(edgesMatrix(:,1))), ', 最大边起点ID
    : ', num2str(max(edgesMatrix(:,1)))]);

nodesTable.Id = nodesTable.Id + 1;
edgesMatrix = edgesMatrix + 1;

disp('修正后节点ID范围:');
disp(['最小节点ID: ', num2str(min(nodesTable.Id)), ', 最大节点ID: ',
    num2str(max(nodesTable.Id))]);
disp(['最小边起点ID: ', num2str(min(edgesMatrix(:,1))), ', 最大边起点ID
    : ', num2str(max(edgesMatrix(:,1)))]);

```

```

% 8. 创建有向图
try
G = digraph(edgesMatrix(:,1), edgesMatrix(:,2));
disp('有向图创建成功!');
disp(['节点数: ', num2str(numnodes(G))]);
disp(['边数: ', num2str(numedges(G))]);
catch ME
error('有向图创建失败: %s', ME.message);
end

% 9. 附加节点属性
disp('===== 附加节点属性 =====');
try
G.Nodes.Id = nodesTable.Id;
[~, idx] = ismember(G.Nodes.Id, nodesTable.Id);
validIdx = idx > 0;

for varName = nodesTable.Properties.VariableNames
varName = varName{1};
if ~strcmp(varName, 'Id')
if isstring(nodesTable.(varName))
G.Nodes.(varName) = strings(numnodes(G), 1);
G.Nodes.(varName)(validIdx) = nodesTable.(varName)(idx(validIdx));
elseif iscell(nodesTable.(varName))
G.Nodes.(varName) = cell(numnodes(G), 1);
G.Nodes.(varName)(validIdx) = nodesTable.(varName)(idx(validIdx));
else
G.Nodes.(varName) = NaN(numnodes(G), 1);
G.Nodes.(varName)(validIdx) = nodesTable.(varName)(idx(validIdx));
end
end
end

disp('节点属性附加成功!');
disp('节点属性字段:');
disp(G.Nodes.Properties.VariableNames);
catch ME
warning(ME.identifier, '%s', ME.message);
end

```

```

end

% 10. 检查图有效性
disp('===== 图结构验证 =====');
% 检查孤立节点
isolatedNodes = find(indegree(G) == 0 & outdegree(G) == 0);
if ~isempty(isolatedNodes)
disp(['发现孤立节点: ', num2str(length(isolatedNodes)), '个']);
disp('孤立节点ID:');
disp(G.Nodes.Id(isolatedNodes));
else
disp('没有发现孤立节点');
end

% 检查环
disp('验证是否为有向无环图 (DAG) ...');
if isdag(G)
disp('验证通过: 该图是有向无环图 (DAG)');
else
try
cycles = allcycles(G);
disp(['警告: 发现环! 环的数量: ', num2str(length(cycles))]);
disp('第一个环的节点:');
disp(cycles{1});
catch
disp('警告: 发现环, 但无法详细列出');
end
end

% 11. 预处理约束信息
disp('===== 预处理约束信息 =====');
constraint_cache = precompute_constraints(G);
disp('约束预处理完成');

% 12. 调用领域搜索贪婪算法
disp('===== 开始领域搜索贪婪算法 =====');
tic;
[best_schedule, best_max_stay, best_fitness, search_stats] =
    domain_greedy_search(G, constraint_cache);

```

```

total_time = toc;

% 13. 显示结果
disp('最优调度序列:');
disp(best_schedule(1:min(20, length(best_schedule)))); % 只显示前20个节点
disp(['... (共', num2str(length(best_schedule)), '个节点)']);
disp(['缓存驻留峰值: ', num2str(best_max_stay)]);
disp(['适应度值: ', num2str(best_fitness)]);

% 14. 验证最终解
[final_max_stay, final_violation, final_fitness] = evaluate_schedule(
    best_schedule, G);
disp('最终验证结果:');
disp(['峰值: ', num2str(final_max_stay), ', 违反约束: ', num2str(
    final_violation)]);

% 15. 性能统计
print_performance_stats(total_time, best_fitness, best_max_stay,
    search_stats);

% 16. 最终验证
validate_final_solution(best_schedule, G);

% 17. 保存结果
save('domain_greedy_results.mat', 'best_schedule', 'best_max_stay', '
    best_fitness', 'total_time', 'search_stats');

writematrix(best_schedule, 'domain_greedy_results.txt');

fprintf('优化完成! 结果已保存到 domain_greedy_results.txt\n');

%% ===== 核心函数实现 =====

%% 领域搜索贪婪算法主函数
function [best_schedule, best_max_stay, best_fitness, stats] =
    domain_greedy_search(G, constraint_cache)
fprintf('开始领域搜索贪婪算法...\n');

```

```

% 初始化统计信息
stats.iterations = 0;
stats.improvements = 0;
stats.neighbors_generated = 0;
stats.best_fitness_history = [];
stats.current_fitness_history = [];

% 生成初始解
fprintf('生成初始解...\n');
current_schedule = greedy_initialization(G, constraint_cache);
[current_max_stay, current_violation, current_fitness] =
    evaluate_schedule(current_schedule, G);

best_schedule = current_schedule;
best_max_stay = current_max_stay;
best_fitness = current_fitness;

fprintf('初始解：峰值=%d, 违反约束=%d, 适应度=%.2f\n', ...
current_max_stay, current_violation, current_fitness);

% 算法参数
max_iterations = 50;
max_no_improve = 10;
tabu_length = 20;
neighborhood_size = 100;

% 初始化禁忌表
tabu_list = [];

no_improve_count = 0;

% 主循环
for iter = 1:max_iterations
stats.iterations = stats.iterations + 1;

fprintf('\n=== 迭代 %d ===\n', iter);

% 生成邻域解
neighbors = generate_neighbors(current_schedule, G, constraint_cache,

```

```

neighborhood_size);
stats.neighbors_generated = stats.neighbors_generated + length(
    neighbors);

% 选择最佳邻域解
[best_neighbor, ~] = select_best_neighbor(neighbors, G, tabu_list,
    current_fitness);

if ~isempty(best_neighbor)
% 更新当前解
current_schedule = best_neighbor.schedule;
current_max_stay = best_neighbor.max_stay;
current_fitness = best_neighbor.fitness;

% 更新禁忌表
tabu_list = update_tabu_list(tabu_list, best_neighbor.move, tabu_length
    );

% 检查是否找到更优解
if best_neighbor.fitness > best_fitness
best_schedule = best_neighbor.schedule;
best_max_stay = best_neighbor.max_stay;
best_fitness = best_neighbor.fitness;
stats.improvements = stats.improvements + 1;
no_improve_count = 0;
fprintf('找到更优解！适应度：%.2f，峰值：%d\n', best_fitness,
    best_max_stay);
else
no_improve_count = no_improve_count + 1;
fprintf('接受邻域解。适应度：%.2f，峰值：%d\n', current_fitness,
    current_max_stay);
end

% 记录历史
stats.best_fitness_history(end+1) = best_fitness;
stats.current_fitness_history(end+1) = current_fitness;
else
no_improve_count = no_improve_count + 1;
fprintf('未找到改进解\n');

```

```

end

% 终止条件检查
if no_improve_count >= max_no_improve
fprintf('连续%d次迭代无改进，终止搜索\n', max_no_improve);
break;
end

% 动态调整搜索策略
if mod(iter, 5) == 0
neighborhood_size = adjust_neighborhood_size(neighborhood_size, stats.
    improvements, iter);
fprintf('调整邻域大小: %d\n', neighborhood_size);
end
end

fprintf('领域搜索完成。总迭代次数: %d, 改进次数: %d\n', stats.
    iterations, stats.improvements);
end

%% 生成邻域解
function neighbors = generate_neighbors(schedule, G, constraint_cache,
    n_neighbors)
neighbors = [];
n_nodes = length(schedule);

% 多种邻域操作
operation_types = {'swap', 'insert', 'reverse'};
weights = [0.5, 0.3, 0.2]; % 操作权重

for i = 1:n_neighbors
% 随机选择操作类型
% 使用轮盘赌选择方法替代randi的权重参数
cum_weights = cumsum(weights);
rand_val = rand();
selected_idx = find(cum_weights >= rand_val, 1);
op_type = operation_types{selected_idx};

switch op_type

```

```

case 'swap'
% 交换两个节点
idx1 = randi(n_nodes);
idx2 = randi(n_nodes);
while idx1 == idx2
idx2 = randi(n_nodes);
end

if is_swap_feasible(schedule, idx1, idx2, G, constraint_cache)
new_schedule = schedule;
new_schedule([idx1, idx2]) = new_schedule([idx2, idx1]);
neighbors = [neighbors, struct('schedule', new_schedule, 'move', ['
    swap_', num2str(idx1), '_', num2str(idx2)])]];
end

case 'insert'
% 插入操作
from_idx = randi(n_nodes);
to_idx = randi(n_nodes);
while from_idx == to_idx
to_idx = randi(n_nodes);
end

if is_insert_feasible(schedule, from_idx, to_idx, G, constraint_cache)
new_schedule = schedule;
node_to_move = new_schedule(from_idx);
new_schedule(from_idx) = [];
new_schedule = [new_schedule(1:to_idx-1), node_to_move, new_schedule(
    to_idx:end)];
neighbors = [neighbors, struct('schedule', new_schedule, 'move', ['
    insert_', num2str(from_idx), '_', num2str(to_idx)])]];
end

case 'reverse'
% 反转子序列
start_idx = randi(floor(n_nodes/2));
end_idx = start_idx + randi(floor(n_nodes/2));
if end_idx > n_nodes
end_idx = n_nodes;

```

```

end

if is_reverse_feasible(schedule, start_idx, end_idx, G,
    constraint_cache)
new_schedule = schedule;
new_schedule(start_idx:end_idx) = fliplr(new_schedule(start_idx:end_idx
    ));
neighbors = [neighbors, struct('schedule', new_schedule, 'move', ['
    reverse_', num2str(start_idx), '_', num2str(end_idx)])]];
end
end
end

% 去除重复的邻域解
if ~isempty(neighbors)
unique_schedules = {};
unique_neighbors = [];

for i = 1:length(neighbors)
schedule_str = mat2str(neighbors(i).schedule);
if ~ismember(schedule_str, unique_schedules)
unique_schedules{end+1} = schedule_str;
unique_neighbors = [unique_neighbors, neighbors(i)];
end
end

neighbors = unique_neighbors;
end
end

%% 选择最佳邻域解
function [best_neighbor, improvement] = select_best_neighbor(neighbors,
    G, tabu_list, current_fitness)
best_neighbor = [];
improvement = false;
best_fitness = -inf;

for i = 1:length(neighbors)
neighbor = neighbors(i);

```

```

% 检查是否在禁忌表中
if is_tabu(neighbor.move, tabu_list)
continue;
end

% 快速评估
[max_stay, violation, fitness] = evaluate_schedule(neighbor.schedule, G
);

% 只考虑可行解
if violation == 0 && fitness > best_fitness
best_fitness = fitness;
best_neighbor = struct(...
'schedule', neighbor.schedule, ...
'max_stay', max_stay, ...
'fitness', fitness, ...
'move', neighbor.move ...
);
end
end

if ~isempty(best_neighbor) && best_neighbor.fitness > current_fitness
improvement = true;
end
end

%% 可行性检查函数
function feasible = is_swap_feasible(schedule, idx1, idx2, G, ~)
feasible = true;

% 检查拓扑约束
node1 = schedule(idx1);
node2 = schedule(idx2);

% 检查前驱关系
preds1 = predecessors(G, node1);
succs1 = successors(G, node1);
preds2 = predecessors(G, node2);

```

```

succs2 = successors(G, node2);

% 节点1不能出现在其前驱之后
for pred = preds1'
pred_pos = find(schedule == pred, 1);
if ~isempty(pred_pos) && pred_pos >= idx2
feasible = false;
return;
end
end

% 节点1不能出现在其后继之前
for succ = succs1'
succ_pos = find(schedule == succ, 1);
if ~isempty(succ_pos) && succ_pos <= idx2
feasible = false;
return;
end
end

% 节点2不能出现在其前驱之后
for pred = preds2'
pred_pos = find(schedule == pred, 1);
if ~isempty(pred_pos) && pred_pos >= idx1
feasible = false;
return;
end
end

% 节点2不能出现在其后继之前
for succ = succs2'
succ_pos = find(schedule == succ, 1);
if ~isempty(succ_pos) && succ_pos <= idx1
feasible = false;
return;
end
end
end
end

```

```

function feasible = is_insert_feasible(schedule, from_idx, to_idx, G,
    ~)
node_id = schedule(from_idx);

% 计算新的位置范围
if to_idx < from_idx
new_pos = to_idx;
old_pos = from_idx;
else
new_pos = to_idx - 1;
old_pos = from_idx;
end

% 检查前驱约束
preds = predecessors(G, node_id);
for pred = preds'
pred_pos = find(schedule == pred, 1);
if ~isempty(pred_pos) && pred_pos >= new_pos
feasible = false;
return;
end
end

% 检查后继约束
succs = successors(G, node_id);
for succ = succs'
succ_pos = find(schedule == succ, 1);
if ~isempty(succ_pos) && succ_pos <= new_pos
feasible = false;
return;
end
end

feasible = true;
end

function feasible = is_reverse_feasible(schedule, start_idx, end_idx, G
    , ~)
% 检查反转后的子序列是否满足所有依赖关系

```

```

reversed_subseq = fliplr(schedule(start_idx:end_idx));

% 检查子序列内部的依赖关系
for i = 1:length(reversed_subseq)
node_id = reversed_subseq(i);

% 检查前驱是否都在子序列中或前面
preds = predecessors(G, node_id);
for pred = preds'
pred_pos = find(schedule == pred, 1);
if ~isempty(pred_pos) && pred_pos >= start_idx && pred_pos <= end_idx
% 前驱在子序列中, 检查位置关系
pred_new_pos = find(reversed_subseq == pred, 1);
if pred_new_pos > i
feasible = false;
return;
end
end
end
end

feasible = true;
end

%% 禁忌表管理
function is_tabu = is_tabu(move, tabu_list)
is_tabu = ismember(move, tabu_list);
end

function new_tabu_list = update_tabu_list(tabu_list, move, max_length)
new_tabu_list = [tabu_list, move];
if length(new_tabu_list) > max_length
new_tabu_list = new_tabu_list(end-max_length+1:end);
end
end

%% 动态调整策略
function new_size = adjust_neighborhood_size(current_size, improvements
, iteration)

```

```

% 根据搜索效果动态调整邻域大小
if improvements < iteration * 0.1
% 改进较少，增大邻域
new_size = min(current_size * 1.2, 500);
else
% 改进较多，保持或减小邻域
new_size = max(current_size * 0.9, 50);
end
end

%% 性能统计函数
function print_performance_stats(total_time, best_fitness,
    best_max_stay, stats)
fprintf('\n===== 性能统计 =====\n');
fprintf('总运行时间: %.2f 秒\n', total_time);
fprintf('总迭代次数: %d\n', stats.iterations);
fprintf('改进次数: %d\n', stats.improvements);
fprintf('邻域解生成总数: %d\n', stats.neighbors_generated);
fprintf('最优适应度: %.2f\n', best_fitness);
fprintf('缓存驻留峰值: %d\n', best_max_stay);
fprintf('平均每次迭代时间: %.3f 秒\n', total_time / stats.iterations);
fprintf('===== \n');

% 绘制收敛曲线
if ~isempty(stats.best_fitness_history)
figure('Position', [100, 100, 800, 400]);
plot(1:length(stats.best_fitness_history), stats.best_fitness_history,
    'b-', 'LineWidth', 2);
hold on;
if ~isempty(stats.current_fitness_history)
plot(1:length(stats.current_fitness_history), stats.
    current_fitness_history, 'r--', 'LineWidth', 1.5);
end
xlabel('迭代次数');
ylabel('适应度值');
legend('历史最优适应度', '当前适应度');
title('领域搜索贪婪算法收敛曲线');
grid on;
set(gca, 'FontSize', 12);

```

```

end
end

%% ===== 辅助函数 =====

%% 约束预处理函数
function constraint_cache = precompute_constraints(G)
fprintf('开始约束预处理...\n');

% 构建拓扑约束矩阵
num_nodes = numnodes(G);
constraint_cache.topology_matrix = zeros(num_nodes, num_nodes);

% 获取所有边
edges = table2array(G.Edges);
for i = 1:size(edges, 1)
u = edges(i, 1);
v = edges(i, 2);
constraint_cache.topology_matrix(u, v) = 1;
end

% 计算传递闭包（用于快速依赖检查）
constraint_cache.reachability = transclosure(G);

% L0缓存冲突关系
L0_types = {'L0A', 'L0B', 'L0C'};
constraint_cache.l0_conflicts = containers.Map();

for type = L0_types
% 收集该类型的所有缓存节点
l0_nodes = G.Nodes(strcmp(G.Nodes.Type, type{1}), :);
buf_ids = [];

for i = 1:height(l0_nodes)
buf_data = l0_nodes.Bufs{i};
if iscell(buf_data)
buf_ids = [buf_ids, buf_data{:}];
else
buf_ids = [buf_ids, buf_data];

```

```

end
end
buf_ids = unique(buf_ids);

constraint_cache.l0_conflicts(type{1}) = buf_ids;
end

% 缓冲区依赖关系
constraint_cache.buffer_dependencies = containers.Map();
cache_nodes = G.Nodes(ismember(G.Nodes.Op, {'ALLOC', 'FREE'}), :);

for i = 1:height(cache_nodes)
    node = cache_nodes(i,:);
    buf_data = node.Bufs{1};
    if iscell(buf_data)
        buf_list = buf_data{:};
    else
        buf_list = buf_data;
    end

    for buf_id = buf_list
        key = num2str(buf_id);
        if ~isKey(constraint_cache.buffer_dependencies, key)
            constraint_cache.buffer_dependencies(key) = struct('alloc_node', [], '
                free_node', []);
        end

        buf_info = constraint_cache.buffer_dependencies(key);
        if strcmp(node.Op{1}, 'ALLOC')
            buf_info.alloc_node = node.Id;
        elseif strcmp(node.Op{1}, 'FREE')
            buf_info.free_node = node.Id;
        end
        constraint_cache.buffer_dependencies(key) = buf_info;
    end
end

% 节点关键度分析
constraint_cache.criticality = zeros(num_nodes, 1);

```

```

for i = 1:num_nodes
% 基于入度和出度计算关键度
constraint_cache.criticality(i) = indegree(G, i) + outdegree(G, i);
end

fprintf('约束预处理完成\n');
end

%% 贪心初始化函数
function schedule = greedy_initialization(G, ~)
fprintf('开始贪心初始化(状态感知动态优先级)...\n');

num_nodes = numnodes(G);
nodes_table = G.Nodes;

% 1. 初始化数据结构
in_degree = indegree(G);
schedule = zeros(1, num_nodes);
scheduled_count = 0;

% 实时追踪L0缓存占用状态
l0_occupied = containers.Map({'L0A', 'L0B', 'L0C'}, {false, false, false});

% 2. 维护一个总的“就绪”节点池
ready_nodes = find(in_degree == 0)';

% 3. 主调度循环
while scheduled_count < num_nodes

if isempty(ready_nodes)
error('无法生成完整拓扑序列！就绪队列为空，但还有 %d 个节点未调度。',
    num_nodes - scheduled_count);
end

best_node_id = -1;
max_priority = -inf;
best_node_idx_in_queue = -1;

```

```

% 4. [核心] 遍历当前所有就绪节点，动态计算优先级并选择最优的一个
for i = 1:length(ready_nodes)
    node_id = ready_nodes(i);
    node_op = nodes_table.Op{node_id};
    node_type = nodes_table.Type{node_id};

    priority = 0; % 基准分

    if strcmp(node_op, 'FREE')
        % 规则1: FREE节点拥有最高优先级
        priority = 1000;
    elseif strcmp(node_op, 'ALLOC')
        is_l0_node = isKey(l0_occupied, node_type);
        if is_l0_node && l0_occupied(node_type)
            % 规则2: 冲突的L0 ALLOC节点，给予极低分，避免选中
            priority = -1000;
        else
            % 规则4: 常规ALLOC节点，优先级较低
            priority = 10;
        end
    else
        % 规则3: 普通计算节点，中等优先级
        priority = 100;
    end

    if priority > max_priority
        max_priority = priority;
        best_node_id = node_id;
        best_node_idx_in_queue = i;
    end
end

% 5. 从就绪池中移除选中的节点，并进行调度
node_to_schedule = best_node_id;
ready_nodes(best_node_idx_in_queue) = [];

scheduled_count = scheduled_count + 1;
schedule(scheduled_count) = node_to_schedule;

```

```

% 6. 更新L0缓存状态
scheduled_op = nodes_table.Op{node_to_schedule};
scheduled_type = nodes_table.Type{node_to_schedule};
if isKey(l0_occupied, scheduled_type)
if strcmp(scheduled_op, 'ALLOC')
l0_occupied(scheduled_type) = true;
elseif strcmp(scheduled_op, 'FREE')
l0_occupied(scheduled_type) = false;
end
end

% 7. 更新后继节点的入度
succs = successors(G, node_to_schedule);
for succ_id = succs'
in_degree(succ_id) = in_degree(succ_id) - 1;
if in_degree(succ_id) == 0
ready_nodes(end+1) = succ_id;
end
end
end

fprintf('贪心初始化完成。生成调度序列长度: %d\n', length(schedule));

% 最终验证
[~, final_violation, ~] = evaluate_schedule(schedule, G);
if final_violation == 0
fprintf('成功生成一个完全可行的初始解! Violation: 0\n');
else
fprintf('警告: 生成的初始解仍有 %d 个约束违反! \n', final_violation);
end

% 验证生成的调度序列
if length(schedule) ~= num_nodes || length(unique(schedule)) ~=
    num_nodes
error('生成的调度序列不完整或包含重复节点');
end

% 检查拓扑约束
edges = table2array(G.Edges);
violation_count = 0;

```

```

for i = 1:size(edges, 1)
    u = edges(i, 1);
    v = edges(i, 2);
    u_pos = find(schedule == u);
    v_pos = find(schedule == v);
    if isempty(u_pos) || isempty(v_pos) || u_pos >= v_pos
        violation_count = violation_count + 1;
    end
end

if violation_count > 0
    fprintf('警告：拓扑约束违反数： %d\n', violation_count);
else
    fprintf('拓扑约束验证通过\n');
end

%% 评估函数
function [max_stay, constraint_violation, fitness] = evaluate_schedule(
    S, G)
persistent id_to_index node_types node_ops node_sizes

num_nodes = length(S);
V_stay = 0;
max_stay = 0;
constraint_violation = 0;

% 初始化快速查找表
if isempty(id_to_index)
    id_to_index = containers.Map(G.Nodes.Id, 1:height(G.Nodes));
    node_types = G.Nodes.Type;
    node_ops = G.Nodes.Op;
    node_sizes = G.Nodes.Size;
end

% 快速完整性检查
if num_nodes ~= height(G.Nodes) || length(unique(S)) ~= num_nodes
    constraint_violation = inf;
    fitness = -inf;
end

```

```

return;
end

% 创建位置映射，用于快速检查拓扑
pos_map = containers.Map('KeyType', 'double', 'ValueType', 'double');
for idx = 1:num_nodes
    pos_map(S(idx)) = idx;
end

% 检查拓扑约束
edges = table2array(G.Edges);
for i = 1:size(edges, 1)
    u = edges(i, 1);
    v = edges(i, 2);
    if pos_map(u) >= pos_map(v)
        constraint_violation = constraint_violation + 1;
    end
end

% 初始化L0缓存占用状态
l0_occupied = containers.Map({'L0A', 'L0B', 'L0C'}, {false, false, false});
valid_cache_types = {'L1', 'UB'}; % 仅计算这两类缓存

for i = 1:num_nodes
    node_id = S(i);
    node_idx = id_to_index(node_id);
    node_type = node_types{node_idx};
    node_op = node_ops{node_idx};
    node_size = node_sizes(node_idx);

    % --- L0 约束检查逻辑 ---
    if isKey(l0_occupied, node_type) % 判断是否是L0A/B/C节点
        if strcmp(node_op, 'ALLOC')
            if l0_occupied(node_type)
                constraint_violation = constraint_violation + 1;
            end
            l0_occupied(node_type) = true; % 标记为占用
        elseif strcmp(node_op, 'FREE')

```

```

l0_occupied(node_type) = false; % 标记为释放
end
end

% --- L1/UB 缓存驻留计算逻辑 ---
if ismember(node_type, valid_cache_types)
if strcmp(node_op, 'ALLOC')
V_stay = V_stay + node_size;
elseif strcmp(node_op, 'FREE')
V_stay = V_stay - node_size;
end
end

if V_stay < 0
% V_stay不应该小于0, 如果发生, 说明FREE在了ALLOC之前
V_stay = 0;
constraint_violation = constraint_violation + 1;
end

if V_stay > max_stay
max_stay = V_stay;
end
end

% 计算适应度
penalty_weight = 1000;
fitness = -max_stay - penalty_weight * constraint_violation;
end

%% 结果验证函数
function validate_final_solution(schedule, G)
fprintf('\n===== 最终解验证 =====\n');

% 检查序列完整性
if length(schedule) ~= height(G.Nodes)
fprintf('错误: 调度序列长度不匹配!\n');
return;
end

```

```

if length(unique(schedule)) ~= height(G.Nodes)
fprintf('错误：调度序列包含重复节点!\n');
return;
end

% 详细评估
[max_stay, violation, fitness] = evaluate_schedule(schedule, G);

fprintf('验证结果:\n');
fprintf('- 序列完整性：通过\n');
fprintf('- 缓存驻留峰值：%d\n', max_stay);
fprintf('- 约束违反数：%d\n', violation);
fprintf('- 适应度值：%.2f\n', fitness);

if violation == 0
fprintf('- 约束满足：全部满足\n');
else
fprintf('- 约束满足：部分违反\n');
end

% 检查拓扑约束详情
edges = table2array(G.Edges);
topo_violations = 0;
for i = 1:size(edges, 1)
u = edges(i, 1);
v = edges(i, 2);
u_pos = find(schedule == u);
v_pos = find(schedule == v);
if u_pos >= v_pos
topo_violations = topo_violations + 1;
fprintf('拓扑违反：节点%d(%d) -> 节点%d(%d)\n', u, u_pos, v, v_pos);
end
end

fprintf('拓扑约束违反总数：%d\n', topo_violations);
fprintf('=====\n');
end

```

code.m

```

%% 问题2：缓存分配与换入换出优化 - ABQPSO完整实现（适配问题二全需求）
% 依据：问题二规则（附录B/C/D/E）+ ABQPSO混合编码优化
clear; clc; close all;

%% ===== 1. 数据加载与预处理（严格适配问题二）
=====

% 1.1 配置文件路径与硬件参数（问题二表1+附录E）
config = struct();
% 1.1.1 文件路径（需根据实际路径修改）
config.nodesPath = 'C:\Users\asus-pc\Desktop\中文赛题ACDEF题\A题\CSV版本\Matmul_Case0_Nodes.csv';
config.edgesPath = 'C:\Users\asus-pc\Desktop\中文赛题ACDEF题\A题\CSV版本\Matmul_Case0_Edges.csv';
config.seqPath = 'C:\Users\asus-pc\Desktop\domain_greedy_results.txt';
% 问题1输出序列
% 1.1.2 硬件缓存容量（问题二表1）
config.cacheCap = struct( ...
    'L1', 4096, ...
    'UB', 1024, ...
    'LOA', 256, ...
    'LOB', 256, ...
    'LOC', 512 ...
);
% 1.1.3 问题二附录常量
config.spillOutPipe = 'MTE3'; % SPILL_OUT执行单元（附录B）
config.spillInPipe = 'MTE2'; % SPILL_IN执行单元（附录B）
config.copyInTag = 'COPY_IN'; % COPY_IN操作标记（附录C.2）
config.alpha = 1.0; % 额外数据搬运量权重（核心目标）
config.beta = 0.1; % 总执行时间权重（次要目标）
config.gamma = 1e6; % 约束违反惩罚权重
config.baseSpillCycles = 150; % SPILL基础周期（附录D）

% 1.2 读取节点数据（附录E格式）
fprintf('=== 加载节点数据（附录E格式） ===\n');
try
% 使用更稳健的方式读取csv文件
opts = detectImportOptions(config.nodesPath);
opts = setvartype(opts, 'string'); % 将所有列先读取为字符串
nodesRaw = readtable(config.nodesPath, opts);

```

```

% 初始化节点结构体
nodes = struct();
numNodes = height(nodesRaw);

% 处理每个字段，确保正确处理缺失值
nodes.Id = zeros(numNodes, 1);
nodes.Op = strings(numNodes, 1);
nodes.BufId = zeros(numNodes, 1);
nodes.Size = zeros(numNodes, 1);
nodes.Type = strings(numNodes, 1);
nodes.Pipe = strings(numNodes, 1);
nodes.Cycles = zeros(numNodes, 1);
nodes.Bufs = cell(numNodes, 1);

for i = 1:numNodes
% Id字段（必须存在）
if ~ismissing(nodesRaw.Id(i)) && ~isempty(nodesRaw.Id{i})
nodes.Id(i) = str2double(nodesRaw.Id{i});
else
error('节点%d的Id字段缺失或为空', i);
end

% Op字段（必须存在）
if ~ismissing(nodesRaw.Op(i)) && ~isempty(nodesRaw.Op{i})
nodes.Op(i) = string(nodesRaw.Op{i});
else
error('节点%d的Op字段缺失或为空', i);
end

% BufId字段（ALLOC/FREE节点必须有）
if ~ismissing(nodesRaw.BufId(i)) && ~isempty(nodesRaw.BufId{i})
nodes.BufId(i) = str2double(nodesRaw.BufId{i});
else
nodes.BufId(i) = NaN; % 用NaN标记缺失值
end

% Size字段（ALLOC/FREE节点必须有）
if ~ismissing(nodesRaw.Size(i)) && ~isempty(nodesRaw.Size{i})

```

```

nodes.Size(i) = str2double(nodesRaw.Size{i});
else
nodes.Size(i) = NaN; % 用NaN标记缺失值
end

% Type字段（ALLOC/FREE节点必须有）
if ~ismissing(nodesRaw.Type(i)) && ~isempty(nodesRaw.Type{i})
nodes.Type(i) = string(nodesRaw.Type{i});
else
nodes.Type(i) = missing; % 用missing标记缺失值
end

% Pipe字段（操作节点必须有）
if ~ismissing(nodesRaw.Pipe(i)) && ~isempty(nodesRaw.Pipe{i}) && ~
    strcmp(nodesRaw.Pipe{i}, '')
nodes.Pipe(i) = string(nodesRaw.Pipe{i});
else
nodes.Pipe(i) = missing; % 保持为missing，在后续函数中处理
end

% Cycles字段（操作节点必须有）
if ~ismissing(nodesRaw.Cycles(i)) && ~isempty(nodesRaw.Cycles{i}) && ~
    strcmp(nodesRaw.Cycles{i}, '')
nodes.Cycles(i) = str2double(nodesRaw.Cycles{i});
else
nodes.Cycles(i) = 0; % 对于缺失的cycles，设为0
end

% Bufs字段（操作节点的输入输出缓冲区列表）
if ~ismissing(nodesRaw.Bufs(i)) && ~isempty(nodesRaw.Bufs{i})
try
nodes.Bufs{i} = str2num(nodesRaw.Bufs{i});
catch
nodes.Bufs{i} = [];
end
else
nodes.Bufs{i} = [];
end
end

```

```

config.originalNodeCount = numNodes;
fprintf('成功加载原节点数: %d\n', config.originalNodeCount);
catch ME
error('节点数据加载失败: %s (需符合附录E CSV格式) \n请检查CSV文件格式是  
否正确, 特别是缺失值的处理。', ME.message);
end

% 1.3 读取依赖边数据 (附录E CSV格式)
fprintf('\n=== 加载依赖边数据 (附录E格式) ===\n');
try
edgesRaw = readtable(config.edgesPath, 'TextType', 'string');
edges = struct();
edges.Start = double(edgesRaw.StartNodeId); % 源节点ID (0索引)
edges.End = double(edgesRaw.EndNodeId); % 目标节点ID (0索引)
fprintf('成功加载依赖边数: %d\n', length(edges.Start));
catch ME
error('边数据加载失败: %s (需符合附录E CSV格式)', ME.message);
end

% 1.4 读取问题1初始调度序列 (附录E schedule.txt格式)
fprintf('\n=== 加载问题1初始调度序列 ===\n');
try
seqRaw = readmatrix(config.seqPath);
config.initSeq = double(seqRaw); % 0索引序列
% 验证序列有效性 (含所有节点+无重复)
if length(unique(config.initSeq)) ~= config.originalNodeCount || ...
~all(ismember(nodes.Id, config.initSeq))
error('初始序列无效: 缺失节点或重复 (需符合问题1输出要求)');
end
fprintf('成功加载初始序列长度: %d\n', length(config.initSeq));
catch ME
error('序列加载失败: %s (需符合附录E schedule.txt格式)', ME.message);
end

% 1.5 预处理: 提取缓冲区核心信息 (问题二优化基础)
fprintf('\n=== 数据预处理 (问题二规则) ===\n');
% 1.5.1 节点ID→索引映射 (快速查找)
nodeIdxMap = containers.Map(nodes.Id, 1:length(nodes.Id));

```

```

% 1.5.2 缓冲区信息: Type/Size/ALLOC/FREE节点 (附录B/C)
bufInfo = getBufInfo(nodes, nodeIdxMap);
config.numBufs = length(keys(bufInfo));
% 1.5.3 标记缓冲区是否被COPY_IN使用 (附录C.2额外数据量计算)
bufIsCopyIn = isBufUsedByCopyIn(nodes, bufInfo, nodeIdxMap, config.
    copyInTag);
% 1.5.4 缓冲区生命周期 (ALLOC→FREE在序列中的位置, 附录B)
bufLifecycle = getBufLifecycle(config.initSeq, bufInfo, nodeIdxMap);
% 1.5.5 BufId→索引映射 (编码用)
bufList = cell2mat(keys(bufInfo));
bufList = double(bufList); % 确保所有元素都是double类型
bufId2Idx = containers.Map(bufList, 1:config.numBufs);

fprintf('提取缓冲区总数: %d | 预处理完成 ==\n\n', config.numBufs);

%% ===== 2. ABQPSO算法参数配置 (适配问题二规模)
%% =====
abqpsoParams = struct();
abqpsoParams.popSize = 50; % 种群规模 (问题二中等规模适配)
abqpsoParams.maxIter = 200; % 最大迭代次数
abqpsoParams.alpha = 0.8; % 收缩扩张系数 (平衡搜索)
abqpsoParams.c1 = 2.0; % 个体学习因子
abqpsoParams.c2 = 2.0; % 社会学习因子
abqpsoParams.spillThresh = 5; % 全局最优未更新阈值 (触发SPILL优化)
abqpsoParams.divThresh = 0.1; % 种群多样性阈值
abqpsoParams.localSearchProb = 0.3; % 局部搜索概率

% 混合编码长度 (问题二决策变量全覆盖)
abqpsoParams.codeLen = struct();
abqpsoParams.codeLen.seq = length(config.initSeq); % 序列调整编码 (原节点顺序)
abqpsoParams.codeLen.offset = config.numBufs; % 缓存分配编码 (BufId→Offset)
abqpsoParams.codeLen.spillFlag = config.numBufs; % SPILL标志编码 (0=无, 1=有)
abqpsoParams.codeLen.spillOffset = config.numBufs; % SPILL偏移编码 (新Offset)

```

```

abqpsoParams.totalCodeLen = sum(struct2array(abqpsoParams.codeLen));

fprintf('=== ABQPSO参数配置 ===\n');
fprintf('种群规模: %d | 迭代次数: %d | 编码长度: %d\n\n', ...
abqpsoParams.popSize, abqpsoParams.maxIter, abqpsoParams.totalCodeLen);

%% ===== 3. ABQPSO核心模块实现（问题二规则适配）
%% =====

% 3.1 初始化粒子群（混合编码+启发式初始化）
fprintf('=== 初始化ABQPSO粒子群 ===\n');
particles = struct();
for i = 1:abqpsoParams.popSize
% 3.1.1 序列编码：问题1序列+局部扰动（保持拓扑序）
seqCode = config.initSeq;
if rand() < 0.3
seqCode = perturbSequence(seqCode, nodes, edges, nodeIdxMap);
end
% 确保seqCode是行向量
if size(seqCode, 1) > 1
seqCode = seqCode';
end

% 3.1.2 缓存分配编码：【性能优化】使用快速随机初始化替换Best-Fit
offsetCode = zeros(1, config.numBufs);
for bIdx = 1:config.numBufs
bufId = bufList(bIdx);
tb = bufInfo(bufId).Type;
cTb = config.cacheCap.(tb);
sizeB = bufInfo(bufId).Size;
% 直接随机分配一个合法的位置。后续的PSO会通过惩罚项来解决重叠问题。
if cTb > sizeB
offsetCode(bIdx) = randi([0, cTb - sizeB]);
else
offsetCode(bIdx) = 0; % 如果缓冲区大小超过容量，先置为0
end
end

% 3.1.3 SPILL编码：初始无SPILL（附录B默认）

```

```

spillFlagCode = zeros(1, config.numBufs);
spillOffsetCode = zeros(1, config.numBufs);

% 3.1.4 组合编码 - 修复维度问题
particles(i).code = [seqCode, offsetCode, spillFlagCode,
    spillOffsetCode];

% 3.1.5 计算适应度（含问题二所有约束）
[fitness, extraData, execTime, penalty] = calculateFitness(particles(i)
    .code, ...
config, abqpsoParams, bufInfo, bufIsCopyIn, bufLifecycle, bufId2Idx,
    bufList, edges, nodeIdXMap, nodes);
particles(i).fitness = fitness;
particles(i).extraData = extraData;
particles(i).execTime = execTime;
particles(i).penalty = penalty;

% 3.1.6 个体最优初始化
particles(i).pbestCode = particles(i).code;
particles(i).pbestFitness = fitness;
end

% 3.2 全局最优初始化
[globalBestFitness, gbIdx] = min([particles.fitness]);
globalBest = struct();
globalBest.code = particles(gbIdx).code;
globalBest.extraData = particles(gbIdx).extraData;
globalBest.execTime = particles(gbIdx).execTime;
globalBest.noUpdateCount = 0;

% 3.3 速度初始化（适配混合编码）
velocities = zeros(abqpsoParams.popSize, abqpsoParams.totalCodeLen);
for i = 1:abqpsoParams.popSize
    velocities(i,:) = randn(1, abqpsoParams.totalCodeLen) * 0.1; % 正态分布
end

fprintf('粒子群初始化完成 | 初始额外数据量: %.0f Bytes ===\n\n',
    globalBest.extraData);

```

```

%% ===== 4. ABQPSO主循环（问题二优化逻辑）
=====
fprintf('== 启动ABQPSO迭代优化（问题二目标） ==\n');
convergenceCurve = zeros(abqpsoParams.maxIter, 4); % [适应度/额外数据/
    时间/惩罚]

for iter = 1:abqpsoParams.maxIter
% 4.1 计算Mbest（平均最优位置）
pbestCodes = reshape([particles.pbestCode], abqpsoParams.totalCodeLen,
    abqpsoParams.popSize)';
Mbest = mean(pbestCodes, 1);

% 4.2 种群迭代更新
for i = 1:abqpsoParams.popSize
% 4.2.1 计算局部吸引子P
phi = rand(1, abqpsoParams.totalCodeLen);
P = phi .* particles(i).pbestCode + (1-phi) .* globalBest.code;

% 4.2.2 速度更新（适配混合编码）
r1 = rand(); r2 = rand();
velocities(i,:) = abqpsoParams.alpha * velocities(i,:) + ...
    abqpsoParams.c1 * r1 * (particles(i).pbestCode - particles(i).code) +
    ...
    abqpsoParams.c2 * r2 * (globalBest.code - particles(i).code);

% 4.2.3 位置更新（分编码类型处理）
newCode = particles(i).code;
% 序列编码更新（交换无依赖节点）
seqRange = 1:abqpsoParams.codeLen.seq;
newCode(seqRange) = updateSeqCode(newCode(seqRange), velocities(i,
    seqRange), ...
    config.initSeq, nodes, edges, nodeIdxMap);
% 缓存分配编码更新（整数截断）
offsetRange = abqpsoParams.codeLen.seq+1 : abqpsoParams.codeLen.seq+
    abqpsoParams.codeLen.offset;
newCode(offsetRange) = updateOffsetCode(newCode(offsetRange),
    velocities(i,offsetRange), ...
    bufInfo, bufList, config.cacheCap);

```

```

% SPILL标志编码更新（二进制阈值）
spillFlagRange = offsetRange(end)+1 : offsetRange(end)+abqpsoParams.
    codeLen.spillFlag;
newCode(spillFlagRange) = updateSpillFlagCode(newCode(spillFlagRange),
    velocities(i,spillFlagRange));
% SPILL偏移编码更新（整数截断）
spillOffsetRange = spillFlagRange(end)+1 : abqpsoParams.totalCodeLen;
newCode(spillOffsetRange) = updateSpillOffsetCode(newCode(
    spillOffsetRange), velocities(i,spillOffsetRange), ...
bufInfo, bufList, config.cacheCap);

% 4.2.4 自适应修复（问题二约束修复）
newCode = adaptiveRepairOperator(newCode, abqpsoParams, config, bufInfo
    , ...
bufLifecycle, bufId2Idx, bufList, edges, nodeIdxMap, nodes);

% 4.2.5 计算新适应度
[newFitness, newExtraData, newExecTime, newPenalty] = calculateFitness(
    newCode, ...
config, abqpsoParams, bufInfo, bufIsCopyIn, bufLifecycle, bufId2Idx,
    bufList, edges, nodeIdxMap, nodes);

% 4.2.6 更新个体最优
if newFitness < particles(i).pbestFitness
particles(i).pbestCode = newCode;
particles(i).pbestFitness = newFitness;
end
particles(i).code = newCode;
particles(i).fitness = newFitness;
particles(i).extraData = newExtraData;
particles(i).execTime = newExecTime;
end

% 4.3 局部搜索（优化缓存碎片+移除冗余SPILL）
if rand() < abqpsoParams.localSearchProb
[newCode, newFitness, newExtraData, newExecTime] = ...
localSearch(globalBest.code, abqpsoParams, config, bufInfo, bufIsCopyIn
    , ...
bufLifecycle, bufId2Idx, bufList, edges, nodeIdxMap, nodes);

```

```

% 仅当局部搜索找到更优解时才更新
if newFitness < globalBestFitness
globalBest.code = newCode;
globalBest.fitness = newFitness;
globalBest.extraData = newExtraData;
globalBest.execTime = newExecTime;
end
end

% 4.4 更新全局最优
[currentBestFitness, currentBestIdx] = min([particles.fitness]);
if currentBestFitness < globalBestFitness
globalBestFitness = currentBestFitness;
globalBest = struct( ...
'code', particles(currentBestIdx).code, ...
'extraData', particles(currentBestIdx).extraData, ...
'execTime', particles(currentBestIdx).execTime, ...
'noUpdateCount', 0 ...
);
else
globalBest.noUpdateCount = globalBest.noUpdateCount + 1;
end

% 4.5 多样性机制（避免局部最优）
popDiversity = calculatePopulationDiversity(particles, abqpsoParams);
if popDiversity < abqpsoParams.divThresh || globalBest.noUpdateCount >=
    abqpsoParams.spillThresh
particles = diversityMechanism(particles, abqpsoParams, config, nodes,
    bufInfo, ...
bufLifecycle, bufId2Idx, bufList, edges, nodeIdXMap);
globalBest.noUpdateCount = 0;
end

% 4.6 记录收敛曲线
convergenceCurve(iter, :) = [globalBestFitness, globalBest.extraData,
    ...
globalBest.execTime, mean([particles.penalty])];

```

```

% 4.7 进度显示
if mod(iter, 20) == 0 || iter == 1
fprintf('迭代%d/%d | 额外数据量: %.0f | 惩罚: %.0f\n', ...
iter, abqpsoParams.maxIter, globalBest.extraData, mean([particles.
    penalty]));
end
end

fprintf('\nABQPSO迭代完成 | 最优额外数据量: %.0f Bytes ===\n',
    globalBest.extraData);

%% ===== 5. 结果解析与输出 (严格符合附录E格式)
=====

fprintf('\n=== 结果解析与输出 (附录E格式) ===\n');
% 5.1 解码最优解: 含SPILL节点的完整调度序列+缓存分配+SPILL操作
[fullSchedule, offsetMap, spillList, spillNodes] = decodeSolution(
    globalBest.code, ...
    config, abqpsoParams, bufInfo, bufLifecycle, bufId2Idx, bufList, edges,
    nodeIdXMap);

% 5.2 核心结果显示
fprintf('1. 问题二核心指标\n');
fprintf('    总额外数据搬运量: %d Bytes\n', globalBest.extraData);
fprintf('    总执行时间: %.0f Cycles\n', globalBest.execTime);
fprintf('    SPILL操作次数: %d\n', length(spillList));

% 5.3 输出附录E格式文件 (Problem2目录)
outputDir = 'Problem2';
if ~exist(outputDir, 'dir'), mkdir(outputDir); end
% 5.3.1 schedule.txt (含SPILL节点的完整序列)
schedulePath = fullfile(outputDir, [extractBefore(config.nodesPath, '
    _Nodes.csv'), '_schedule.txt']);
writematrix(fullSchedule, schedulePath, 'Delimiter', ' ');
% 5.3.2 memory.txt (BufId:Offset)
memoryPath = fullfile(outputDir, [extractBefore(config.nodesPath, '
    _Nodes.csv'), '_memory.txt']);
fid = fopen(memoryPath, 'w');
for b = keys(offsetMap) '

```

```

b = b{1};
fprintf(fid, '%d:%d\n', b, offsetMap(b));
end
fclose(fid);

% 5.3.3 spill.txt (BufId:NewOffset, 无则空文件)
spillPath = fullfile(outputDir, [extractBefore(config.nodesPath, '
    _Nodes.csv'), '_spill.txt']);
fid = fopen(spillPath, 'w');
for i = 1:length(spillList)
    fprintf(fid, '%d:%d\n', spillList(i).BufId, spillList(i).NewOffset);
end
fclose(fid);

fprintf('2. 附录E格式文件已生成: \n');
fprintf('    - %s\n    - %s\n    - %s\n', schedulePath, memoryPath,
    spillPath);

% 5.4 收敛曲线绘制
figure('Position', [100, 100, 1200, 400]);
subplot(1,3,1); plot(1:abqpsoParams.maxIter, convergenceCurve(:,1), 'b-
    ', 'LineWidth', 2);
xlabel('迭代次数'); ylabel('适应度'); title('ABQPso适应度收敛'); grid
    on;
subplot(1,3,2); plot(1:abqpsoParams.maxIter, convergenceCurve(:,2), 'r-
    ', 'LineWidth', 2);
xlabel('迭代次数'); ylabel('额外数据量(Bytes)'); title('额外数据量收敛'
    ); grid on;
subplot(1,3,3); plot(1:abqpsoParams.maxIter, convergenceCurve(:,4), 'g-
    ', 'LineWidth', 2);
xlabel('迭代次数'); ylabel('平均惩罚'); title('约束满足度收敛'); grid
    on;

fprintf('\n问题二完整流程结束 ===\n');

%% ===== 6. 核心辅助函数 =====

% 6.1 提取缓冲区信息 (附录B/C)
function bufInfo = getBufInfo(nodes, nodeIdxMap)

```

```

% 创建Map容器，键为BufId，值为结构体
bufInfo = containers.Map('KeyType', 'double', 'ValueType', 'any');

for i = 1:length(nodes.Id)
% 【修改】使用 () 访问字符串数组
op = nodes.Op(i);
if strcmp(op, 'ALLOC') || strcmp(op, 'FREE')
b = double(nodes.BufId(i)); % 确保键是double类型且非NaN

% 检查BufId是否有效
if isnan(b)
continue; % 跳过无效的BufId
end

% 如果键不存在，创建新结构体
if ~isKey(bufInfo, b)
bufInfo(b) = struct(...
'Type', '', ... % 使用 "" 初始化字符串
'Size', 0, ...
'allocNode', -1, ...
'freeNode', -1);
end

% 获取当前结构体副本
currentBuf = bufInfo(b);

% 更新结构体字段
if strcmp(op, 'ALLOC')
currentBuf.Type = nodes.Type(i);
currentBuf.Size = nodes.Size(i);
currentBuf.allocNode = nodes.Id(i);
else
currentBuf.freeNode = nodes.Id(i);
end

% 将修改后的结构体存回Map
bufInfo(b) = currentBuf;
end
end

```

```

% 过滤无效缓冲区（无ALLOC/FREE）
bufKeys = keys(bufInfo);
for i = 1:length(bufKeys)
    b = bufKeys{i};
    if bufInfo(b).allocNode == -1 || bufInfo(b).freeNode == -1
        remove(bufInfo, b);
    end
end
end

% 6.2 标记缓冲区是否被COPY_IN使用（附录C.2）
function bufIsCopyIn = isBufUsedByCopyIn(nodes, bufInfo, nodeIdxMap,
    copyInTag)
% 创建与bufInfo相同键的映射，初始值为false
bufKeys = keys(bufInfo);
bufIsCopyIn = containers.Map('KeyType', 'double', 'ValueType', 'logical',
    '');
for i = 1:length(bufKeys)
    bufIsCopyIn(bufKeys{i}) = false;
end

% 标记被COPY_IN使用的缓冲区
for i = 1:length(nodes.Id)
    % 【修改】使用 () 访问字符串数组
    if strcmp(nodes.Op(i), copyInTag)
        % 【注意】nodes.Bufs是cell array，这里使用 {} 是正确的
        bufs = nodes.Bufs{i};
        for j = 1:length(bufs)
            b = bufs(j);
            if isKey(bufIsCopyIn, b)
                bufIsCopyIn(b) = true;
            end
        end
    end
end
end

% 6.3 计算缓冲区生命周期（附录B）

```

```

function bufLifecycle = getBufLifecycle(seq, bufInfo, nodeIdxMap)
% 创建Map容器，键类型应与bufInfo一致（double）
bufLifecycle = containers.Map('KeyType', 'double', 'ValueType', 'any');

% 创建节点ID到序列位置的映射
nodePosMap = containers.Map(seq, 1:length(seq)); % 节点ID→序列位置

% 遍历所有缓冲区
bufKeys = keys(bufInfo);
for i = 1:length(bufKeys)
    b = bufKeys{i};
    allocNode = bufInfo(b).allocNode;
    freeNode = bufInfo(b).freeNode;

    % 检查ALLOC/FREE节点是否在序列中
    if ~isKey(nodePosMap, allocNode) || ~isKey(nodePosMap, freeNode)
        error('缓冲区%d的ALLOC/FREE节点不在序列中', b);
    end

    % 获取节点在序列中的位置
    allocPos = nodePosMap(allocNode);
    freePos = nodePosMap(freeNode);

    % 检查生命周期有效性
    if allocPos >= freePos
        error('缓冲区%d生命周期无效：ALLOC在FREE之后', b);
    end

    % 存储生命周期信息
    bufLifecycle(b) = [allocPos, freePos];
end
end

% 6.4 序列扰动（保持拓扑序）
function newSeq = perturbSequence(seq, nodes, edges, nodeIdxMap)
newSeq = seq;
idx1 = randi(length(seq)); idx2 = randi(length(seq));
if idx1 == idx2, return; end
node1 = newSeq(idx1); node2 = newSeq(idx2);

```

```

% 检查无依赖边（避免破坏拓扑序）
hasEdge = any(edges.Start == node1 & edges.End == node2) || ...
any(edges.Start == node2 & edges.End == node1);
if ~hasEdge
newSeq([idx1, idx2]) = newSeq([idx2, idx1]);
end
end

% 6.5 最佳适应缓存分配（减少碎片）
function bestOffset = findBestFitOffset(minOff, maxOff, sizeB,
    offsetCode, bufList, bufInfo, bIdx, bufLifecycle, seq)
bestOffset = minOff;
minGap = maxOff;

% 获取当前缓冲区的信息
currentBufId = bufList(bIdx);
currentType = bufInfo(currentBufId).Type;
currentLifecycle = bufLifecycle(currentBufId);
currentAlloc = currentLifecycle(1);
currentFree = currentLifecycle(2);

% 遍历所有可能偏移，找最小碎片间隙
for off = minOff:maxOff - sizeB + 1
% 检查与已分配缓冲区的重叠（同类型+生命周期重叠）
overlap = false;

for j = 1:bIdx-1
otherBufId = bufList(j);
if strcmp(bufInfo(otherBufId).Type, currentType)
otherLifecycle = bufLifecycle(otherBufId);
otherAlloc = otherLifecycle(1);
otherFree = otherLifecycle(2);

% 检查生命周期是否重叠
if ~(currentFree < otherAlloc || otherFree < currentAlloc)
% 检查地址空间是否重叠
otherOff = offsetCode(j);
otherSize = bufInfo(otherBufId).Size;
if ~(off + sizeB - 1 < otherOff || otherOff + otherSize - 1 < off)

```

```

overlap = true;
break;
end
end
end
end

if ~overlap
gap = (off + sizeB) - off; % 实际占用间隙
if gap < minGap
minGap = gap;
bestOffset = off;
end
end
end
end

% 6.6 适应度计算（问题二目标+全约束）
function [fitness, extraData, execTime, penalty] = calculateFitness(
    code, config, params, bufInfo, bufIsCopyIn, bufLifecycle, bufId2Idx
    , bufList, edges, nodeIdXMap, nodes)
% 解码：含SPILL节点的完整解
[fullSeq, offsetMap, spillList, spillNodes, newEdges] = decodeSolution(
    code, config, params, bufInfo, bufLifecycle, bufId2Idx, bufList,
    edges, nodeIdXMap);

% 1. 计算总额外数据搬运量（附录C.2）
extraData = 0;
for i = 1:length(spillList)
b = spillList(i).BufId;
sizeB = bufInfo(b).Size;
if bufIsCopyIn(b)
extraData = extraData + sizeB; % 仅SPILL_IN（附录C.2情况2）
else
extraData = extraData + 2 * sizeB; % SPILL_OUT+IN（情况1）
end
end

% 2. 计算总执行时间（附录C.1三步法）

```

```

execTime = calculateExecTime(fullSeq, nodes, spillNodes, newEdges,
    nodeIdXMap, config);

% 3. 约束违反惩罚（问题二全约束）
penalty = 0;
% 3.1 地址合法性（不超过缓存容量）
for b = keys(offsetMap) '
b = b{1};
tb = bufInfo(b).Type;
cTb = config.cacheCap.(tb);
sizeB = bufInfo(b).Size;
off = offsetMap(b);
if off < 0 || off + sizeB > cTb
penalty = penalty + config.gamma;
end
end
% 3.2 缓存非重叠（同时驻留无重叠）
[~, overlapCnt] = checkNoOverlap(fullSeq, offsetMap, bufInfo,
    bufLifecycle, spillList, bufId2Idx, config);
penalty = penalty + config.gamma * overlapCnt;
% 3.3 L0缓存约束（L0A/B/C各最多1个驻留，问题1说明）
[~, l0Cnt] = checkL0Constraint(fullSeq, offsetMap, bufInfo,
    bufLifecycle, spillList, bufId2Idx);
penalty = penalty + config.gamma * l0Cnt;
% 3.4 拓扑依赖（含SPILL节点）
[~, topoCnt] = checkTopoOrder(fullSeq, newEdges, nodeIdXMap, spillNodes
    , nodes);
penalty = penalty + config.gamma * topoCnt;
% 3.5 缓存复用依赖（附录C.2步骤2：复用地址需FREE→ALLOC依赖）
[~, reuseCnt] = checkCacheReuseDependency(offsetMap, bufInfo,
    bufLifecycle, newEdges);
penalty = penalty + config.gamma * reuseCnt;

% 4. 总适应度（最小化）
fitness = config.alpha * extraData + config.beta * execTime + penalty;
end

% 6.7 解码函数（核心：生成SPILL节点+完整序列+依赖边）
function [fullSchedule, offsetMap, spillList, spillNodes, newEdges] =

```

```

    decodeSolution(code, config, params, bufInfo, bufLifecycle,
        bufId2Idx, bufList, edges, nodeIdXMap)
% 1. 解析基础编码
seqCode = code(1:params.codeLen.seq);
offsetCode = code(params.codeLen.seq+1 : params.codeLen.seq+params.
    codeLen.offset);
spillFlagCode = code(params.codeLen.seq+params.codeLen.offset+1 :
    params.codeLen.seq+params.codeLen.offset+params.codeLen.spillFlag);
spillOffsetCode = code(params.codeLen.seq+params.codeLen.offset+params.
    codeLen.spillFlag+1 : end);

% 2. 缓存分配映射 (BufId→Offset) - 修复键类型问题
offsetMap = containers.Map('KeyType', 'double', 'ValueType', 'double');
for i = 1:params.codeLen.offset
    bufId = double(bufList(i)); % 确保bufId是double类型
    offsetMap(bufId) = offsetCode(i);
end

% 3. SPILL操作解析 (生成SPILL节点+列表, 附录B)
spillList = [];
spillNodes = struct('Id', [], 'Op', [], 'Pipe', [], 'Cycles', [], 'Bufs',
    [], []);
currentSpillId = config.originalNodeCount; % SPILL节点Id从原节点数开始
    (附录B)
nodePosMap = containers.Map(seqCode, 1:length(seqCode)); % 原节点→位置

for i = 1:params.codeLen.spillFlag
    if spillFlagCode(i) == 1
        bufId = bufList(i);
        newOffset = spillOffsetCode(i);
        lifecycle = bufLifecycle(bufId);
        allocPos = lifecycle(1);
        freePos = lifecycle(2);

% 3.1 生成SPILL_OUT节点 (附录B属性)
        spillOutCycles = bufInfo(bufId).Size * 2 + config.baseSpillCycles;

        spillNodes(end+1) = struct( ...
            'Id', currentSpillId, ...

```

```

'Op', "SPILL_OUT", ...
'Pipe', string(config.spillOutPipe), ...
'Cycles', spillOutCycles, ...
'Bufs', {{bufId}} ...
);
spillOutId = currentSpillId;
currentSpillId = currentSpillId + 1;

% 3.2 生成SPILL_IN节点（附录B属性）
spillInCycles = bufInfo(bufId).Size * 2 + config.baseSpillCycles; % 附录D通用
spillNodes(end+1) = struct( ...
'Id', currentSpillId, ...
'Op', "SPILL_IN", ...
'Pipe', string(config.spillInPipe), ...
'Cycles', spillInCycles, ...
'Bufs', {{bufId}} ...
);
spillInId = currentSpillId;
currentSpillId = currentSpillId + 1;

% 3.3 记录SPILL操作（BufId:NewOffset，附录E）
spillList(end+1) = struct( ...
'BufId', bufId, ...
'SpillOutId', spillOutId, ...
'SpillInId', spillInId, ...
'NewOffset', newOffset, ...
'SpillOutPos', allocPos + 1, ...
'SpillInPos', freePos - 1 ...
);
offsetMap(bufId) = newOffset; % 更新为SPILL后的新偏移
end
end
spillNodes(1) = []; % 移除初始化的空struct

% 4. 构建含SPILL节点的完整调度序列
fullSchedule = seqCode;
% 按位置插入SPILL节点（SPILL_OUT在ALLOC后，SPILL_IN在FREE前）
for i = length(spillList):-1:1 % 倒序插入避免位置偏移

```

```

soPos = spillList(i).SpillOutPos;
siPos = spillList(i).SpillInPos;
fullSchedule = [fullSchedule(1:soPos-1); spillList(i).SpillOutId;
    fullSchedule(soPos:end)];
fullSchedule = [fullSchedule(1:siPos); spillList(i).SpillInId;
    fullSchedule(siPos+1:end)];
end

% 5. 更新依赖边（附录B规则）
newEdges = [edges.Start, edges.End]; % 原边
for i = 1:length(spillList)
    bufId = spillList(i).BufId;
    allocNode = bufInfo(bufId).allocNode;
    freeNode = bufInfo(bufId).freeNode;
    soId = spillList(i).SpillOutId;
    siId = spillList(i).SpillInId;

    % 5.1 基础依赖边（ALLOC→SPILL_OUT→SPILL_IN→FREE）
    newEdges = [newEdges; allocNode, soId];
    newEdges = [newEdges; soId, siId];
    newEdges = [newEdges; siId, freeNode];

    % 5.2 操作节点依赖边（已执行→SPILL_OUT，SPILL_IN→未执行）
    lifecycle = bufLifecycle(bufId);
    allocPos = lifecycle(1);
    freePos = lifecycle(2);
    for nodeId = seqCode'
        nodeIdx = nodeIdxMap(nodeId);
        if ismember(bufId, nodes.Bufs{nodeIdx}) % 使用该缓冲区的操作节点
            nodePos = nodePosMap(nodeId);
            if nodePos > allocPos && nodePos < freePos % 检查节点是否在生命周期内
                newEdges = [newEdges; nodeId, soId];
                newEdges = [newEdges; siId, nodeId];
            end
        end
    end
end

% 6. 去重依赖边

```

```

newEdges = unique(newEdges, 'rows');
end

% 6.8 序列编码更新
function newSeqCode = updateSeqCode(seqCode, velocity, initSeq, nodes,
    edges, nodeIdxMap)
newSeqCode = seqCode;
velAbs = abs(velocity);
[~, swapIdx] = sort(velAbs, 'descend');
if length(swapIdx) >= 2
    idx1 = swapIdx(1); idx2 = swapIdx(2);
    node1 = newSeqCode(idx1); node2 = newSeqCode(idx2);
    % 检查无依赖
    hasEdge = any(edges.Start == node1 & edges.End == node2) || ...
        any(edges.Start == node2 & edges.End == node1);
    if ~hasEdge
        newSeqCode([idx1, idx2]) = newSeqCode([idx2, idx1]);
    end
end
end

% 6.9 缓存分配编码更新
function newOffsetCode = updateOffsetCode(offsetCode, velocity, bufInfo
    , bufList, cacheCap)
newOffsetCode = offsetCode;
for i = 1:length(offsetCode)
    bufId = bufList(i);
    tb = bufInfo(bufId).Type;
    cTb = cacheCap.(tb);
    sizeB = bufInfo(bufId).Size;
    % 整数截断+合法性约束
    newOff = round(offsetCode(i) + velocity(i));
    newOff = max(0, min(newOff, cTb - sizeB));
    newOffsetCode(i) = newOff;
end
end

% 6.10 SPILL标志编码更新
function newSpillFlagCode = updateSpillFlagCode(spillFlagCode, velocity

```

```

    )
newSpillFlagCode = spillFlagCode;
for i = 1:length(spillFlagCode)
if abs(velocity(i)) > 0.5
newSpillFlagCode(i) = 1 - spillFlagCode(i); % 翻转
end
end
end

% 6.11 SPILL偏移编码更新
function newSpillOffsetCode = updateSpillOffsetCode(spillOffsetCode,
    velocity, bufInfo, bufList, cacheCap)
newSpillOffsetCode = spillOffsetCode;
for i = 1:length(spillOffsetCode)
bufId = bufList(i);
tb = bufInfo(bufId).Type;
cTb = cacheCap.(tb);
sizeB = bufInfo(bufId).Size;
newOff = round(spillOffsetCode(i) + velocity(i));
newOff = max(0, min(newOff, cTb - sizeB));
newSpillOffsetCode(i) = newOff;
end
end

% 6.12 自适应修复算子（问题二约束修复）
function repairedCode = adaptiveRepairOperator(code, params, config,
    bufInfo, bufLifecycle, bufId2Idx, bufList, edges, nodeIdXMap, nodes
)
repairedCode = code;
% 解码当前解
[fullSeq, offsetMap, spillList, spillNodes, newEdges] = decodeSolution(
    repairedCode, config, params, bufInfo, bufLifecycle, bufId2Idx,
    bufList, edges, nodeIdXMap);

% 1. 修复缓存重叠（优先SPILL低成本缓冲区）
[~, overlapCnt] = checkNoOverlap(fullSeq, offsetMap, bufInfo,
    bufLifecycle, spillList, bufId2Idx, config);
while overlapCnt > 0
overlapBufs = getOverlapBufs(fullSeq, offsetMap, bufInfo, bufLifecycle,

```

```

        spillList, bufId2Idx, config);
if isempty(overlapBufs) % 如果没有找到重叠的buf, 则跳出循环避免死循环
break;
end
% 计算SPILL成本 (额外数据量)
spillCost = [];
bufIsCopyIn = isBufUsedByCopyIn(nodes, bufInfo, nodeIdxMap, config.
    copyInTag);
for b = overlapBufs
sizeB = bufInfo(b).Size;
if bufIsCopyIn(b)
spillCost = [spillCost, sizeB];
else
spillCost = [spillCost, 2*sizeB];
end
end
% 选择成本最小的缓冲区SPILL
[~, minIdx] = min(spillCost);
targetBuf = overlapBufs(minIdx);
bIdx = bufId2Idx(targetBuf);
% 更新SPILL编码
spillFlagPos = params.codeLen.seq + params.codeLen.offset + bIdx;
repairedCode(spillFlagPos) = 1;
% 更新SPILL偏移 (重新分配)
tb = bufInfo(targetBuf).Type;
cTb = config.cacheCap.(tb);
newOff = findBestFitOffset(0, cTb - bufInfo(targetBuf).Size, bufInfo(
    targetBuf).Size, ...
repairedCode(params.codeLen.seq+1:params.codeLen.seq+params.codeLen.
    offset), ...
bufList, bufInfo, bIdx, bufLifecycle, repairedCode(1:params.codeLen.seq
    ));
spillOffsetPos = params.codeLen.seq + params.codeLen.offset + params.
    codeLen.spillFlag + bIdx;
repairedCode(spillOffsetPos) = newOff;
% 重新检查
[fullSeq, offsetMap, spillList, ~, newEdges] = decodeSolution(
    repairedCode, config, params, bufInfo, bufLifecycle, bufId2Idx,
    bufList, edges, nodeIdxMap);

```

```

[~, overlapCnt] = checkNoOverlap(fullSeq, offsetMap, bufInfo,
    bufLifecycle, spillList, bufId2Idx, config);
end

% 2. 修复拓扑依赖
[isTopoValid, ~] = checkTopoOrder(fullSeq, newEdges, nodeIdxMap,
    spillNodes, nodes);
if ~isTopoValid
    repairedSeq = repairTopoOrder(fullSeq, newEdges);
% 更新序列编码
repairedCode(1:params.codeLen.seq) = repairedSeq(ismember(repairedSeq,
    config.initSeq)); % 仅保留原节点
end
end

% 6.13 局部搜索（优化碎片+移除冗余SPILL）
function [newCode, newFitness, newExtraData, newExecTime] = localSearch
    (code, params, config, bufInfo, bufIsCopyIn, bufLifecycle,
    bufId2Idx, bufList, edges, nodeIdxMap, nodes)
newCode = code;
% 解码
[fullSeq, offsetMap, spillList, ~, ~] = decodeSolution(newCode, config,
    params, bufInfo, bufLifecycle, bufId2Idx, bufList, edges,
    nodeIdxMap);

% 1. 缓存偏移微调（减少碎片）
offsetRange = params.codeLen.seq+1 : params.codeLen.seq+params.codeLen.
    offset;
for i = 1:params.codeLen.offset
    if rand() < 0.2
        bufId = bufList(i);
        tb = bufInfo(bufId).Type;
        cTb = config.cacheCap.(tb);
        sizeB = bufInfo(bufId).Size;
        currentOff = newCode(offsetRange(i));
% 微调±1（合法性）
newOff = currentOff + (randi(3)-2); % -1, 0, or 1
newOff = max(0, min(newOff, cTb - sizeB));
newCode(offsetRange(i)) = newOff;

```

```

end
end

% 2. 移除冗余SPILL (无重叠则可移除)
[~, overlapCnt] = checkNoOverlap(fullSeq, offsetMap, bufInfo,
    bufLifecycle, spillList, bufId2Idx, config);
if overlapCnt == 0
spillFlagRange = params.codeLen.seq+params.codeLen.offset+1 : params.
    codeLen.seq+params.codeLen.offset+params.codeLen.spillFlag;
for i = 1:params.codeLen.spillFlag
if newCode(spillFlagRange(i)) == 1 && rand() < 0.5
newCode(spillFlagRange(i)) = 0; % 尝试移除SPILL
end
end
end

% 3. 重新计算适应度
[newFitness, newExtraData, newExecTime, ~] = calculateFitness(newCode,
    ...
    config, params, bufInfo, bufIsCopyIn, bufLifecycle, bufId2Idx, bufList,
    edges, nodeIdXMap, nodes);
end

% 6.14 种群多样性计算
function diversity = calculatePopulationDiversity(particles, params)
totalDist = 0; cnt = 0;
for i = 1:params.popSize
for j = i+1:params.popSize
dist = sum(particles(i).code ~= particles(j).code) / params.
    totalCodeLen;
totalDist = totalDist + dist;
cnt = cnt + 1;
end
end
if cnt == 0
diversity = 0;
else
diversity = totalDist / cnt;
end

```

```

end

% 6.15 多样性机制
function newParticles = diversityMechanism(particles, params, config,
    nodes, bufInfo, bufLifecycle, bufId2Idx, bufList, edges, nodeIdxMap
    )
newParticles = particles;
[~, sortedFitnessIdx] = sort([particles.fitness], 'descend');
resetCnt = round(params.popSize * 0.2); % 重置20%最差的粒子
resetIdx = sortedFitnessIdx(1:resetCnt);

for i = resetIdx
% 重新初始化编码
seqCode = config.initSeq;
if rand() < 0.5 % 增加扰动概率
seqCode = perturbSequence(seqCode, nodes, edges, nodeIdxMap);
end
% 最佳适应缓存分配
offsetCode = zeros(1, params.codeLen.offset);
for bIdx = 1:params.codeLen.offset
bufId = bufList(bIdx);
tb = bufInfo(bufId).Type;
cTb = config.cacheCap.(tb);
sizeB = bufInfo(bufId).Size;
offsetCode(bIdx) = findBestFitOffset(0, cTb - sizeB, sizeB, offsetCode,
    ...
    bufList, bufInfo, bIdx, bufLifecycle, seqCode);
end
spillFlagCode = zeros(1, params.codeLen.spillFlag);
spillOffsetCode = zeros(1, params.codeLen.spillOffset);
% 更新粒子
newParticles(i).code = [seqCode, offsetCode, spillFlagCode,
    spillOffsetCode];
bufIsCopyIn = isBufUsedByCopyIn(nodes, bufInfo, nodeIdxMap, config.
    copyInTag);
[newFitness, newExtraData, newExecTime, newPenalty] = calculateFitness(
    newParticles(i).code, ...
    config, params, bufInfo, bufIsCopyIn, bufLifecycle, bufId2Idx, bufList,
    edges, nodeIdxMap, nodes);

```

```

newParticles(i).fitness = newFitness;
newParticles(i).extraData = newExtraData;
newParticles(i).execTime = newExecTime;
newParticles(i).penalty = newPenalty;
newParticles(i).pbestCode = newParticles(i).code;
newParticles(i).pbestFitness = newFitness;
end
end

% 6.16 缓存非重叠检查 【性能优化：事件驱动的扫描线算法】
function [isValid, violationCnt] = checkNoOverlap(fullSeq, offsetMap,
    bufInfo, bufLifecycle, spillList, bufId2Idx, config)
isValid = true;
violationCnt = 0;

cacheTypes = fieldnames(config.cacheCap)';
for type_idx = 1:length(cacheTypes)
    tb = cacheTypes{type_idx};

    events = [];
    % 1. 创建事件列表
    for b_cell = keys(offsetMap)
        b = b_cell{1};
        if strcmp(bufInfo(b).Type, tb)
            lifecycle = bufLifecycle(b);
            % 检查生命周期是否有效
            if length(lifecycle) < 2 || lifecycle(1) < 1
                continue;
            end

            % 正常的ALLOC-FREE周期
            events = [events; struct('pos', lifecycle(1), 'type', 1, 'bufId', b)];
            % 1 for ALLOC
            events = [events; struct('pos', lifecycle(2), 'type', -1, 'bufId', b)];
            % -1 for FREE
        end
    end

    % 如果没有该类型的缓冲区，跳过

```

```

if isempty(events)
continue;
end

% 2. 按位置对事件排序
[~, sortIdx] = sort([events.pos]);
events = events(sortIdx);

% 3. 扫描线
residentBufs = containers.Map('KeyType','double','ValueType','any');
for i = 1:length(events)
event = events(i);

if event.type == 1 % ALLOC事件
newBufId = event.bufId;
newOff = offsetMap(newBufId);
newSize = bufInfo(newBufId).Size;

% 检查与当前所有驻留缓冲区重叠
residentKeys = keys(residentBufs);
for j = 1:length(residentKeys)
resId = residentKeys{j};
resOff = offsetMap(resId);
resSize = bufInfo(resId).Size;
if ~(newOff + newSize <= resOff || resOff + resSize <= newOff)
violationCnt = violationCnt + 1;
end
end
residentBufs(newBufId) = 1; % 加入驻留集合
else % FREE事件
if isKey(residentBufs, event.bufId)
remove(residentBufs, event.bufId); % 移出驻留集合
end
end
end
end

if violationCnt > 0
isValid = false;

```

```

end
end

% 6.17 L0缓存约束检查 【性能优化：事件驱动】
function [isValid, violationCnt] = checkL0Constraint(fullSeq, offsetMap
    , bufInfo, bufLifecycle, spillList, bufId2Idx)
isValid = true;
violationCnt = 0;
l0Types = {'L0A', 'LOB', 'LOC'};

for type_idx = 1:length(l0Types)
    tb = l0Types{type_idx};

    events = [];
    % 1. 创建事件列表
    for b_cell = keys(offsetMap)
        b = b_cell{1};
        if strcmp(bufInfo(b).Type, tb)
            lifecycle = bufLifecycle(b);
            if length(lifecycle) < 2 || lifecycle(1) < 1
                continue;
            end
            events = [events; struct('pos', lifecycle(1), 'type', 1)]; % 1 for
                ALLOC
            events = [events; struct('pos', lifecycle(2), 'type', -1)]; % -1 for
                FREE
        end
    end

    if isempty(events)
        continue;
    end

    % 2. 排序
    [~, sortIdx] = sort([events.pos]);
    events = events(sortIdx);

    % 3. 扫描
    residentCnt = 0;

```

```

for i = 1:length(events)
    residentCnt = residentCnt + events(i).type;
    if residentCnt > 1
        % 在下一个事件发生前，都处于违规状态
        violationCnt = violationCnt + 1;
    end
end
end

if violationCnt > 0
    isValid = false;
end
end

% 6.18 拓扑依赖检查（含SPILL节点）
function [isValid, violationCnt] = checkTopoOrder(fullSeq, newEdges,
    nodeIdxMap, spillNodes, nodes)
isValid = true;
violationCnt = 0;
nodePosMap = containers.Map(fullSeq, 1:length(fullSeq));

% 检查所有依赖边
for i = 1:size(newEdges, 1)
    u = newEdges(i, 1);
    v = newEdges(i, 2);
    if ~isKey(nodePosMap, u) || ~isKey(nodePosMap, v)
        % 如果边中的节点不在当前序列中（可能由于序列修复等原因），也视为一种违
        % 规
        isValid = false;
        violationCnt = violationCnt + 1;
        continue;
    end
    if nodePosMap(u) >= nodePosMap(v)
        isValid = false;
        violationCnt = violationCnt + 1;
    end
end
end
end

```

```

% 6.19 缓存复用依赖检查（附录C.2步骤2）
function [isValid, violationCnt] = checkCacheReuseDependency(offsetMap,
    bufInfo, bufLifecycle, newEdges)
isValid = true;
violationCnt = 0;
bufList = cell2mat(keys(offsetMap)');

% 查找地址复用的缓冲区对
for i = 1:length(bufList)-1
b1 = bufList(i);
for j = i+1:length(bufList)
b2 = bufList(j);
if strcmp(bufInfo(b1).Type, bufInfo(b2).Type) && offsetMap(b1) ==
    offsetMap(b2)
% 复用同一地址，需b1.FREE → b2.ALLOC依赖
freeNode1 = bufInfo(b1).freeNode;
allocNode2 = bufInfo(b2).allocNode;
hasReuseEdge = any(newEdges(:,1) == freeNode1 & newEdges(:,2) ==
    allocNode2);
if ~hasReuseEdge
isValid = false;
violationCnt = violationCnt + 1;
end
end
end
end
end

% 6.20 总执行时间计算（附录C.1三步法）
function execTime = calculateExecTime(fullSeq, nodes, spillNodes,
    newEdges, nodeIdxMap, config)
% 1. 初始化单元执行时间线
unitTimeLine = containers.Map();

% 【优化】更健壮地收集所有唯一的pipe名称
validNodePipes = nodes.Pipe(~ismissing(nodes.Pipe));
validSpillPipes = strings(0);
if ~isempty(spillNodes) && isfield(spillNodes, 'Pipe')
spillPipes = [spillNodes.Pipe];

```

```

validSpillPipes = spillPipes(~ismissing(spillPipes));
end
allPipes = unique([validNodePipes; validSpillPipes]);

for i = 1:length(allPipes)
    pipe = char(allPipes(i)); % Map的键需要是char
    unitTimeLine(pipe) = 0; % 单元当前可用时间
end

% 2. 初始化节点开始/结束时间
nodeStartTime = containers.Map('KeyType', 'double', 'ValueType', 'double');
nodeEndTime = containers.Map('KeyType', 'double', 'ValueType', 'double');
allNodeIds = fullSeq;
for i = 1:length(allNodeIds)
    nodeId = allNodeIds(i);
    nodeStartTime(nodeId) = 0;
    nodeEndTime(nodeId) = 0;
end

% 3. 按序列计算时间
for i = 1:length(fullSeq)
    nodeId = fullSeq(i);

    % 获取节点属性
    pipe = ""; % 使用字符串类型
    cycles = 0;
    isOriginalNode = isKey(nodeIdxMap, nodeId);

    if isOriginalNode
        idx = nodeIdxMap(nodeId);

        % 【修改】使用 () 访问字符串数组
        currentPipe = nodes.Pipe(idx);
        if ~ismissing(currentPipe)
            pipe = currentPipe;
        end
    end
end

```

```

currentCycles = nodes.Cycles(idx);
if ~ismissing(currentCycles) && ~isnan(currentCycles)
cycles = double(currentCycles);
else
cycles = 0; % 缓存管理节点周期为0
end
else
spillIdx = find([spillNodes.Id] == nodeId, 1);
if ~isempty(spillIdx)
currentPipe = spillNodes(spillIdx).Pipe;
if ~ismissing(currentPipe)
pipe = currentPipe;
end
cycles = double(spillNodes(spillIdx).Cycles);
end
end

% 计算开始时间
startT = 0;
preNodes = newEdges(newEdges(:,2) == nodeId, 1);
if ~isempty(preNodes)
maxPreEndTime = 0;
for p_idx = 1:length(preNodes)
p = preNodes(p_idx);
if isKey(nodeEndTime, p)
maxPreEndTime = max(maxPreEndTime, nodeEndTime(p));
end
end
startT = maxPreEndTime;
end

% 如果节点占用硬件单元，考虑资源约束
if pipe ~= ""
pipe_char = char(pipe);
if isKey(unitTimeLine, pipe_char)
startT = max(startT, unitTimeLine(pipe_char));
% 更新单元时间线
unitTimeLine(pipe_char) = startT + cycles;
end

```

```

end

% 更新节点时间
nodeStartTime(nodeId) = startT;
nodeEndTime(nodeId) = startT + cycles;
end

% 总执行时间：所有节点结束时间的最大值
if isempty(nodeEndTime) || isempty(values(nodeEndTime))
execTime = 0;
else
execTime = max(cell2mat(values(nodeEndTime)));
end
end

% 6.21 获取重叠缓冲区
function overlapBufs = getOverlapBufs(fullSeq, offsetMap, bufInfo,
    bufLifecycle, spillList, bufId2Idx, config)
overlapBufs = [];
nodePosMap = containers.Map(fullSeq, 1:length(fullSeq));
bufList = cell2mat(keys(offsetMap)');
combinedLifecycle = containers.Map(bufLifecycle.keys, bufLifecycle.
    values);

% 合并SPILL生命周期
for i = 1:length(spillList)
b = spillList(i).BufId;
soPos = nodePosMap(spillList(i).SpillOutId);
siPos = nodePosMap(spillList(i).SpillInId);
lifecycle = bufLifecycle(b);
allocPos = lifecycle(1);
freePos = lifecycle(2);
combinedLifecycle(b) = [allocPos, soPos, siPos, freePos];
end

% 查找重叠
for pos = 1:length(fullSeq)
cacheTypes = fieldnames(config.cacheCap)';
for type_idx = 1:length(cacheTypes)

```

```

tb = cacheTypes{type_idx};
residentBufs = [];
for buf_idx = 1:length(bufList)
b = bufList(buf_idx);
if strcmp(bufInfo(b).Type, tb)
lifecycle = combinedLifecycle(b);
isResident = false;
if (length(lifecycle)==2 && pos>=lifecycle(1)&&pos<=lifecycle(2)) ||
...
(length(lifecycle)==4 && ((pos>=lifecycle(1)&&pos<lifecycle(2)) || (pos
>lifecycle(3)&&pos<=lifecycle(4))))
isResident = true;
end
if isResident
residentBufs = [residentBufs, b];
end
end
end
for i = 1:length(residentBufs)-1
b1 = residentBufs(i);
off1 = offsetMap(b1);
size1 = bufInfo(b1).Size;
for j = i+1:length(residentBufs)
b2 = residentBufs(j);
off2 = offsetMap(b2);
size2 = bufInfo(b2).Size;
if ~(off1 + size1 <= off2 || off2 + size2 <= off1)
overlapBufs = [overlapBufs, b1, b2];
end
end
end
end
end
overlapBufs = unique(overlapBufs);
end

% 6.22 修复拓扑依赖
function repairedSeq = repairTopoOrder(seq, newEdges)
repairedSeq = seq;

```

```

nodePosMap = containers.Map(repairedSeq, 1:length(repairedSeq));

max_repair_iter = length(repairedSeq)^2;
iter_count = 0;
changed = true;
while changed && iter_count < max_repair_iter
    changed = false;
    iter_count = iter_count + 1;
    for i = 1:size(newEdges, 1)
        u = newEdges(i, 1);
        v = newEdges(i, 2);

        is_u_in_seq = isKey(nodePosMap, u);
        is_v_in_seq = isKey(nodePosMap, v);

        if is_u_in_seq && is_v_in_seq
            if nodePosMap(u) > nodePosMap(v)
                posU = nodePosMap(u);
                posV = nodePosMap(v);
                repairedSeq([posU, posV]) = repairedSeq([posV, posU]);
                % 更新位置映射
                nodePosMap(u) = posV;
                nodePosMap(v) = posU;
                changed = true;
            end
        end
    end
    if iter_count >= max_repair_iter
        warning('拓扑修复达到最大迭代次数，可能存在无法解决的依赖环。');
    end
end

```